

PROFILE OF THE UNIVERSITY



MAULANA MAZHARUL HAQUE UNIVERSITY

ARABIC & PERSIAN, PATNA

Maulana Mazharul Haque Arabic and Persian University (MMHAPU) in Patna is a leading university in Bihar. Its primary mission is to empower students, with a special focus on those from underprivileged or marginalized backgrounds. The university works to break down barriers by making quality mainstream education more accessible and inclusive for them. Beyond academic knowledge, it also dedicates itself to the overall personal growth of its students. It teaches them about important global values and ethical principles, encouraging them to develop their skills and talents to the fullest. Ultimately, the university aims to shape students into well-rounded, responsible, and forward-thinking citizens who can contribute positively to society.

Guidance

Mr. Abdul Basit Sir

*Ass. Lecturer Dept of BCA
AIC, Bihar Sharif Nalanda*

Director

Dr. Md Mushtaque Ahmad

AIC, Bihar Sharif Nalanda

A Project Report on

Electricity Bill Management System (EBMS)

Submitted as part of the
requirements for the

BACHELOR OF COMPUTER APPLICATION

By

Md Modassir

Under The Guidance Of

Mr. Abdul Basit Sir

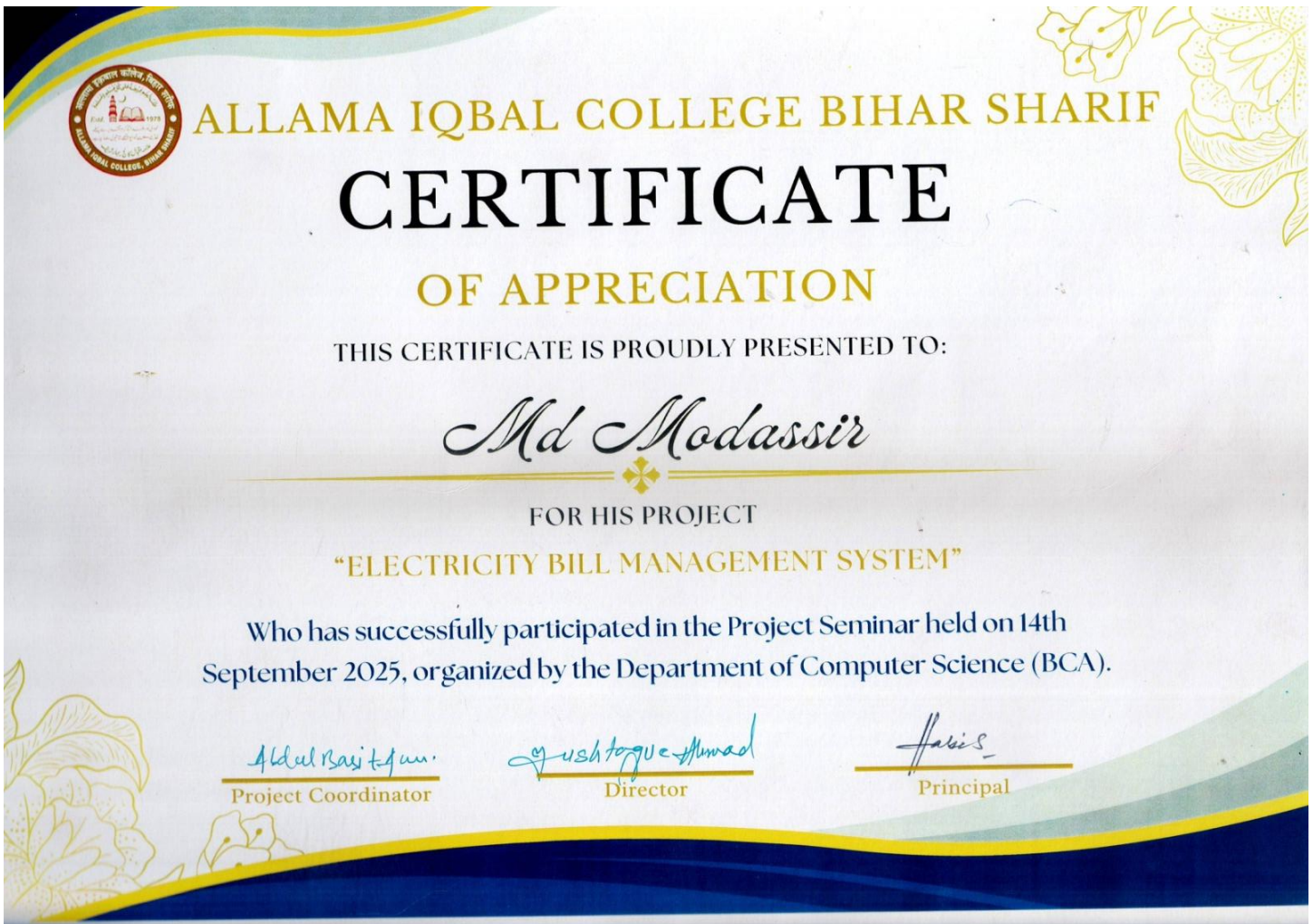


DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

ALLAMA IQBAL COLLEGE, BIHARSHARIF - 803101 BIHAR

ESTD 1978

CERTIFICATE OF COMPLETION



I am Md Modassir, a student of "Bachelor of Computer Application (BCA)". I am doing this course from Allama Iqbal College, a recognized study center of "Maulana Mazharul Haque Arabic & Persian University". I declare that I did not submit this project Report to any award. It makes only for partial fulfillment of BCA. I tried my level best in making this project according to my knowledge.

Name: Md Modassir

University Roll No: 231680200007

Registration No: 16202218667

ALLAMA IQBAL COLLEGE

BIHAR SHARIF



DECLARATION

I hereby declare that the project entitled "ELECTRICITY BILL MANAGEMENT SYSTEM" Submitted to the MAULANA MAZHARUL HAQUE ARABIC & PERSIAN UNIVERSITY, PATNA for the partial fulfillment of the requirement of the award of degree of BCA is a record of the original work done by me under the supervision & Guidance of ABDUL BASIT ANSARI.

Name of candidate: Md Modassir

University Roll.No: 231680200007

Registration No: 16202218667

Sign. Of Guidance

Foreword

In an age defined by digital innovation and the pursuit of seamless convenience, the way we manage essential services must evolve. We recognized that the traditional processes surrounding electricity billing—often static and one-sided—were ripe for transformation. Our goal was to move beyond simple transactions and create a dynamic, interactive experience for everyone involved.

It is with great pride that we introduce our new Electricity Bill Management System. This platform represents a fundamental shift in how we serve our community. For our customers, it is a gateway to unprecedented convenience and control. You can now effortlessly view and pay your bills online, check the real-time status of your payments and requests, and communicate directly with us to provide feedback or resolve concerns. This system is designed to give you immediate insight and command over your account, turning what was once a routine chore into a simple, integrated part of your digital life.

For our administration and vendor partners, this system is the cornerstone of a more efficient and responsive operation. It provides a comprehensive toolkit for managing user accounts, generating accurate bills, and overseeing the entire service lifecycle with clarity and precision. This centralized approach allows us to serve you more effectively, ensuring that every interaction is handled with speed and accuracy.

Looking ahead, our vision extends far beyond the current feature set. We are actively exploring the integration of smart meter analytics for personalized consumption insights, AI-powered outage prediction and management, and expanded payment options like cryptocurrency. Our roadmap also includes a dedicated mobile application and open API frameworks for seamless integration with smart home ecosystems, empowering you with even greater control and efficiency in the future.

This system is more than a technological upgrade; it is a reflection of our commitment to building a transparent, efficient, and collaborative relationship with you. We have built this platform with a clear focus on empowering our users and streamlining our operations, setting a new standard for service in the utilities sector.

We are excited to invite you to explore this new system and experience the difference for yourself. Welcome to a smarter, more connected way to manage your electricity needs.

Preface

The journey to develop this Electricity Bill Management System began with a simple yet powerful observation: the relationship between a utility provider and its customers should be a dialogue, not a monologue, built on a foundation of absolute trust and efficiency. For years, we understood that the traditional process was not just inconvenient, but also susceptible to inefficiencies and potential malpractices. This project was born from a commitment to foster a transparent, corruption-free, and time-saving environment for everyone involved.

This platform is the culmination of countless hours of collaboration between our engineers, customer service teams, and administrators. Our development process was guided by a user-centric philosophy, ensuring that every feature solves a real-world problem. A core motivation was to empower you, the user, in two critical ways. First, by enabling direct online payments, the system creates a clear, digital audit trail, eliminating cash handling and any potential for unauthorized intermediaries. This ensures that every rupee you pay is accurately accounted for, directly enhancing accountability and integrity.

Second, we recognized the immense value of your time. This system is designed to give you complete control over your time management. You can now manage all your electricity-related tasks—paying bills, checking status, raising complaints—instantly, from anywhere, at any time. This eliminates the need for travel, long queues, and waiting for office hours, freeing up your valuable time for what truly matters to you.

While the foreword outlines the system's broad capabilities, this preface serves to highlight these fundamental benefits of transparency and convenience. We see this launch as a starting point for a more efficient and trustworthy relationship. We invite you to experience the peace of mind that comes with secure, direct transactions and the freedom of seamless time management.

Key benefits at a Glance:

- Enhanced Transparency: Digital trail for all payments and requests.
- Time Efficiency: 24/7 access from any device, saving you hours.
- Direct Accountability: Secure, user-to-provider financial transactions
- Instant Resolution: Real-time tracking of complaints and requests.
- User Empowerment: Full control over your account and consumption data.

Acknowledgement

We would like to express our sincere gratitude and appreciation to everyone who contributed to the successful development and implementation of the Electricity Bill Management System. This project would not have been possible without the unwavering support and guidance of our project advisors *Mr. Abdul Basit Sir*, whose vision and encouragement were instrumental from conception to completion. We are also deeply thankful to our dedicated team of developers, designers, and testers for their relentless effort, expertise, and commitment to excellence. A special note of thanks extends to our colleagues in the customer service and administration departments, whose invaluable insights shaped the system's practical functionality. Finally, we acknowledge the patience and valuable feedback from our initial users, which has been crucial in refining the platform. This achievement is a testament to the collaboration and hard work of all Involved.

CURRENT SYSTEM

CHAPTER 1: INTRODUCTION.....	9 - 10
1.1 OVERVIEW:.....	9
1.1.1 OBJECTIVE:	9
1.1.2 PURPOSE & SCOPE:.....	10
CHAPTER 2: TECHNOLOGIES USED	11
CHAPTER 3: UNDERSTANDING THE SYSTEM.....	12
3.1 CUREENT SYSTEM:	12
3.2 PROPOSED SOLUTION:.....	12
CHAPTER 4: SDLC “THE RECIPE”	13 - 88
4.1 REQUIREMENT GATHERING & ANALYSIS:.....	14
4.2 SYSTEM DESIGN:.....	14 - 21
4.2.1 CONCEPTUAL DESIGN:.....	14 - 21
4.2.1.1 : ENTITY RELATIONSHIP DIAGRAM.....	15
4.2.1.2 : DATA FLOW DIAGRAM.....	16 - 19
4.2.1.3 : SYSTEM ARCHITECTURE DESIGN.....	20
4.3 TECHNOLOGY STACK & IMPLEMENTATION:.....	21 - 77
4.3.1 TECHNOLOGY STACK SPECIFICATION:.....	21
4.3.1.1 : BACKEND TECHNOLOGIES.....	21 - 22
4.3.1.2 : DATABASE MANAGEMENT SYSTEM.....	22
4.3.1.3 : DEVELOPMENT TOOLS & ENVIRONMENT.....	22
4.3.2 IMPLEMENTATION (CODING):.....	22 - 77
4.4 TESTING:.....	78 - 87
4.5 MAINTENANCE:.....	88

CHAPTER 1: INTRODUCTION

1.1. OVERVIEW:

The "Electricity Bill Management System" project aims to automatically generate electricity bills with all applicable charges. The manual system currently in use is time-consuming and ineffective, making the billing process difficult and Complicated.

My goal is to create a system that computerizes some of the Electricity Board's work. This includes generating monthly bills, tracking energy use, and storing customer and payment records. I built it using Java for the application and MySQL for the database.

MySQL Server is a relational database application that allows multiple users to securely access and manage large amounts of data at the same time.

1.1.1 OBJECTIVE:

- Easy maintenance of database in overall project.
- Adopting security measures, by maintaining the login of username and password.
- Reducing data redundancy.
- Reduce workload of staff.
- Reduce the delay in processing time.
- Provide user-friendliness in all possible ways.
- Store data in a centralized location to reduce redundancy and increase consistency
- Digitalization of bill management
- Digital management of customers records

1.1.2 **PURPOSE AND SCOPE:**

Problems with the Manual System

The existing manual process for managing electricity bills is slow and inefficient because it relies on physical paperwork. This leads to several major problems:

Slow Data Access: Finding a specific customer's file or payment history takes a long time, causing delays in generating bills and answering customer queries.

Difficulty Managing Large Volumes of Data: Storing and organizing large amounts of paper records is a cumbersome and space-consuming task.

High Risk of Errors: Manual data entry is prone to mistakes in calculations and record-keeping, leading to incorrect bills.

Data Duplication: The same customer information might be recorded in multiple ledgers, creating confusion and inconsistency.

Difficulty in Updating Records: Correcting an error or updating a customer's address is a tedious process that requires finding and altering physical documents.

Lack of a Centralized System: There is no single source of truth. Getting a complete view of a customer's history or generating reports is very difficult.

Poor Data Security: Paper records are vulnerable to damage from fire, water, or wear and tear, and are less secure against unauthorized access.

Inefficient Reporting: Generating summary reports, like total revenue or energy consumption patterns, is a slow and manual effort.

Solution: The New Automated System

The purpose of this project is to overcome these problems by automating the entire system. By creating a centralized digital database, we aim to:

Increase Speed and Efficiency: Retrieve any customer's information and generate bills instantly.

Improve Accuracy: Automate calculations to eliminate human error in billing.

Ensure Data Consistency: Maintain a single, updated record for each customer to prevent duplication.

Simplify Updates: Allow information to be modified quickly and easily within the software.

Enhance Security: Protect data with secure logins and digital backups.

Enable Easy Reporting: Generate various reports (e.g., unpaid bills, monthly revenue) with a single click.

CHAPTER 2: TECHNOLOGIES USED

This chapter describes the key technologies used to build this project, explaining why they were chosen and how they contribute to the system.

- **JAVA** is chosen as the primary programming language for this project. It is a powerful, high-level language known for its reliability and widespread use in building large-scale applications.

Key advantages of using Java include:

Platform Independence: Java follows the "write once, run anywhere" (WORA) principle. Code is compiled into bytecode, which can run on any device equipped with a Java Virtual Machine (JVM), making the application highly portable across Windows, Linux, and macOS.

Object-Oriented Programming (OOP): Java is fundamentally object-oriented. This approach helps in creating modular, flexible, and reusable code, which makes the software easier to manage, extend, and maintain.

- **Class:** A class is like a blueprint or template. For example, a *Student* class would define attributes like *id* and *name*, and actions like *calculateGrade()*.
- **Object:** An object is a specific instance created from a class. Using the *Student* blueprint, we can create objects like *student1* (with id 101, name "Alice") and *student2* (with id 102, name "Bob").

Strong Memory Management: Java handles memory allocation and deallocation automatically through its built-in Garbage Collector. This prevents common errors like memory leaks and makes the development process more robust.

Version: This project has been developed using **Java SE 17** or **Java 11 (LTS version recommended for stability)**. This version provides long-term support, ensuring the application remains stable and maintainable.

- **MYSQL SERVER** is used as the database system for storing and retrieving the project's data. The JDBC (Java Database Connectivity) API is used to connect the Java application to the MySQL database.

Key advantages of using MySQL include:

Relational Database Management System (RDBMS): MySQL organizes data into tables (rows and columns), which is an intuitive and efficient way to manage structured information.

Structured Query Language (SQL): It uses SQL, which is the standard language for communicating with relational databases. SQL is powerful and versatile for tasks like inserting, querying, updating, and deleting data.

Reliability and Performance: MySQL is renowned for its speed, reliability, and ease of use. It can handle a large amount of data efficiently, making it suitable for a wide range of applications.

CHAPTER 3: UNDERSTANDING THE SYSTEM

System analysis is the initial and crucial step in creating or improving a system, where the main goal is to thoroughly understand how the current system works and identify its problems. This is done by gathering detailed information through methods like interviews and questionnaires, which helps the system analyst diagnose issues and figure out what needs to be changed. The process involves a lot of communication between the developers and the people who use the system every day. After studying all aspects of the existing system, the analyst proposes solutions; these proposals are then carefully compared to the old system, reviewed by the users, and adjusted until everyone is satisfied. This phase also includes a preliminary study to assess the project's feasibility, which helps in planning the next steps for an effective development process.

3.1 CURRENT SYSTEM: PROBLEMS AND LIMITATIONS

This section describes the manual way of managing bills, which is slow and inefficient. It relies on paper, requires a lot of staff time for calculations and filing, and lacks data security. Because information is on paper, it's hard for managers to track progress or make decisions easily. The main problems are that it is time-consuming, prone to errors, and insecure.

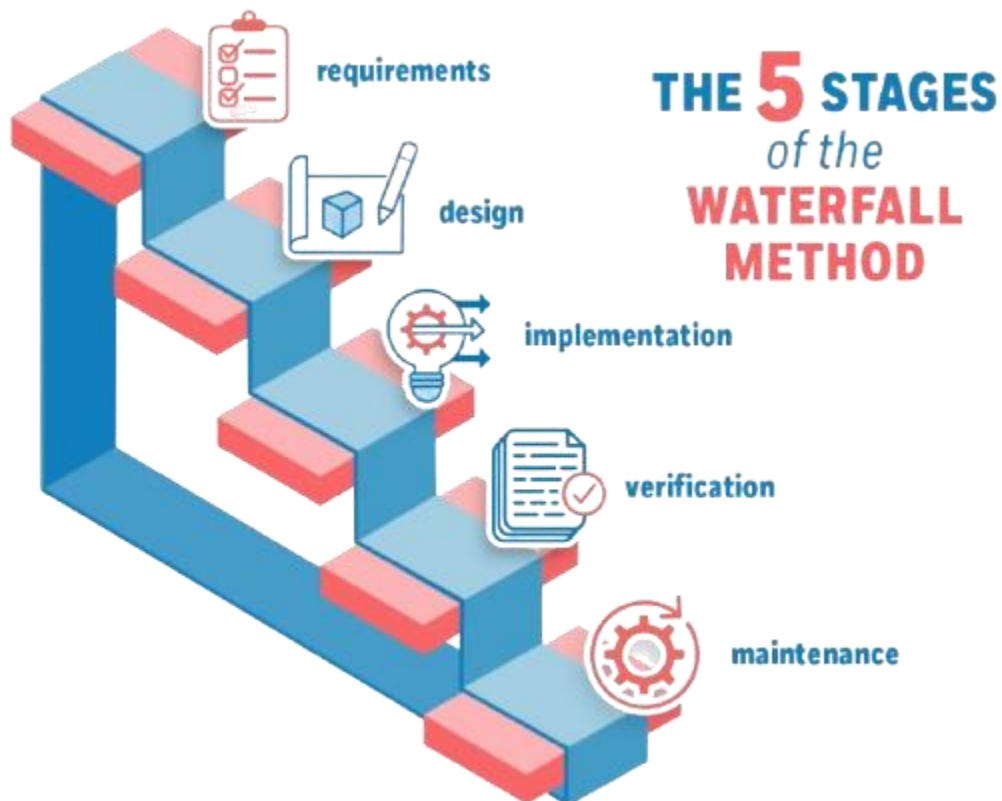
- **Connects Cause and Effect:** It links the "offline" and "paper-based" nature of the system directly to the problem of corruption.
- **Uses Strong Language:** Phrases like "opens the door to corruption" and "unofficial charges" clearly describe the problem without being overly accusatory.
- **Highlights Human Impact:** Mentioning that it's "draining" to stand in queues adds a human element, showing the system's toll on people's time and energy.
- **Ties to Management:** It ends by stating that managers can't easily see or stop these problems, which reinforces the need for a transparent, computerized system.

3.2 PROPOSED SOLUTION: AUTOMATED BILL MANAGEMENT

The new system replaces the manual process with a simple computer application. This software will securely manage all billing data, drastically reducing paper use and manual entry. It automatically handles calculations, ensuring accuracy and saving time. The system also gives managers clear oversight and control. Overall, it leads to greater efficiency, better security, and a more user-friendly experience.

CHAPTER 4: SDLC “THE RECIPE”

The development of the Electricity Bill Management System was guided by a structured Software Development Life Cycle (SDLC) to ensure a systematic and efficient approach to project completion. We adopted the **Waterfall model**, which provided a clear, phase-by-phase framework. The process commenced with the **Requirement Gathering and Analysis** phase, where we conducted detailed discussions to understand the needs of consumers, billing clerks, and administrators. This was followed by the **System Design** phase, where we created comprehensive conceptual models, including Entity-Relationship Diagrams (ERDs) and Data Flow Diagrams (DFDs), to blueprint the system's architecture and data relationships. Upon design approval, we proceeded to the **Implementation** phase, where the actual coding was done using Java. The subsequent Testing phase involved rigorous unit, integration, and system testing to identify and rectify defects, ensuring reliability in calculating tariffs, generating bills, and managing user data. Finally, the project will conclude with a **Deployment** plan and an outline for **Maintenance** to ensure the system's long-term viability and ease of future updates. This sequential SDLC approach ensured that each stage's deliverables were met methodically, leading to a robust and well-documented final product.



The Waterfall Model of the Software Development Life Cycle (SDLC)

4.1 Requirement Gathering & Analysis

The primary objective of this initial phase was to meticulously define the capabilities and constraints of the Electricity Bill Management System. Through a process of brainstorming and analysis of existing systems, we identified key stakeholders: **Users** and **Administrators**.

The core functional requirements elicited include:

- The system shall allow consumers to view and download their monthly bills.
- The system shall automatically calculate charges based on slab-based tariff rates defined by an admin.
- The system shall maintain a history of all payments received.

Key non-functional requirements include data security through password hashing and a user-friendly interface for ease of use. These requirements were documented in a Software Requirements Specification (SRS) which served as the foundational guide for the subsequent system design phase.

4.2 System Design

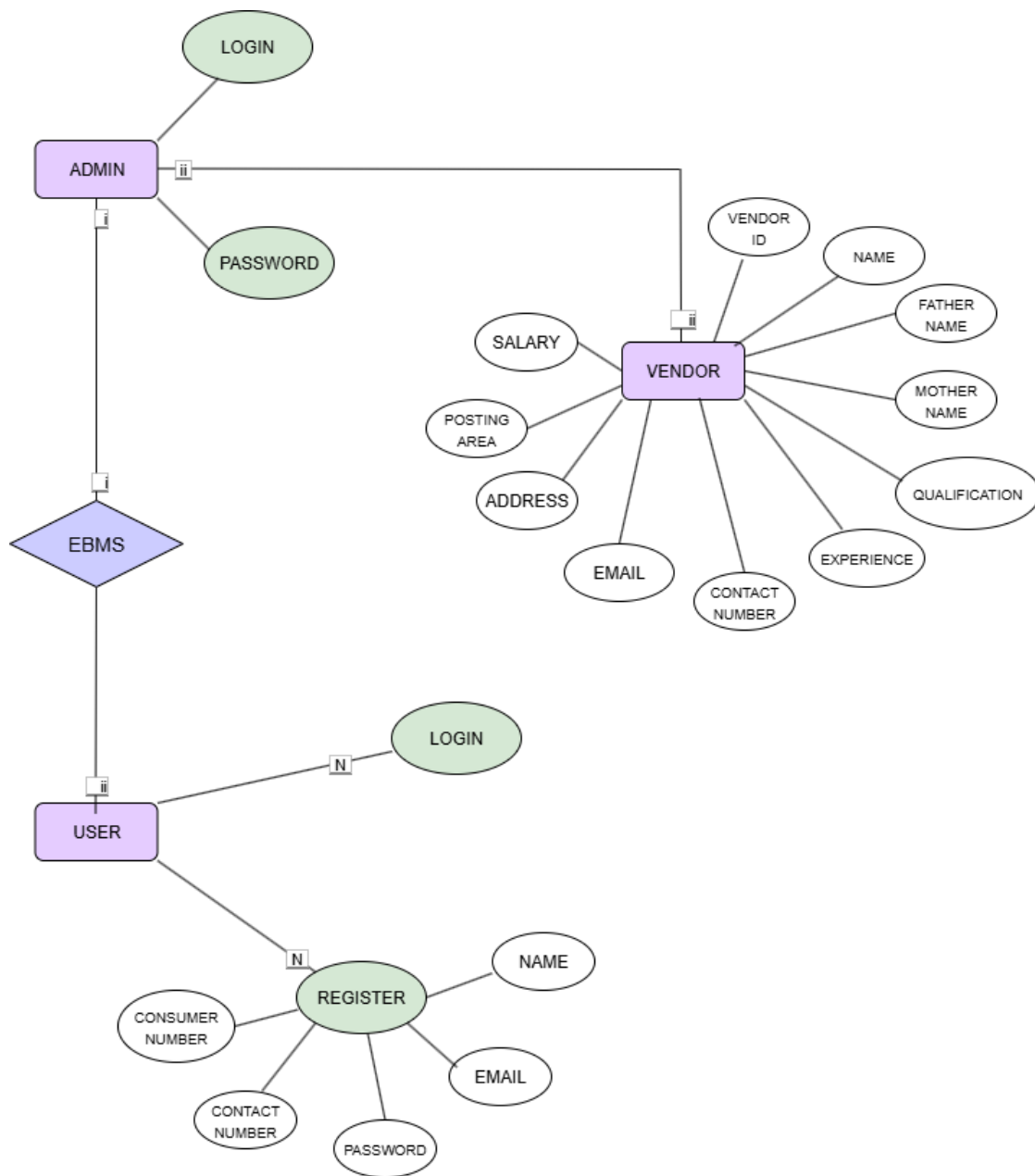
The System Design phase translates the requirements for the Electricity Bill Management System into a actionable development blueprint. This phase focuses on defining the system's architecture, data structures, and component interactions before any code is written. Key activities include creating conceptual models like Entity-Relationship Diagrams (ERDs) to design the database schema and Data Flow Diagrams (DFDs) to map process logic, alongside selecting the technology stack and designing the user interface. This deliberate planning step is essential for ensuring a smooth, efficient, and well-organized implementation process.

4.2.1 Conceptual Design

The conceptual design represents the logical structure of the system, independent of any specific database management system or implementation technology. Its primary goal is to create a high-level, user-centric model that focuses on the key data entities, their attributes, and the fundamental relationships between them. This step is crucial for establishing a shared understanding of the system's domain among all stakeholders—including clients, analysts, and developers—before committing to technical details. For the Electricity Bill Management System, conceptual modeling allows us to visually define core elements like Consumers, Bills, and Payments, ensuring that the database schema will accurately reflect the real-world processes of electricity billing.

- ERD (Entity Relationship Diagram)
- DFD (Data Flow Diagram)
- UCD (Use Case Diagram)

4.2.1.1 Entity-Relationship Diagram (ERD)



The enhanced Entity-Relationship Diagram (ERD) for the Electricity Bill Management System now comprehensively models all key stakeholders and their interactions. The diagram introduces three crucial user roles: Admin, Vendor, and User (Consumer), each with distinct attributes and responsibilities.

The Admin entity, with its login and password attributes, serves as the system administrator responsible for overall management, user account oversight, and system configuration. The Vendor entity represents field

personnel or employees, capturing detailed personal and professional information including qualification, posting_area, and salary, which enables efficient workforce management and area-wise assignment tracking.

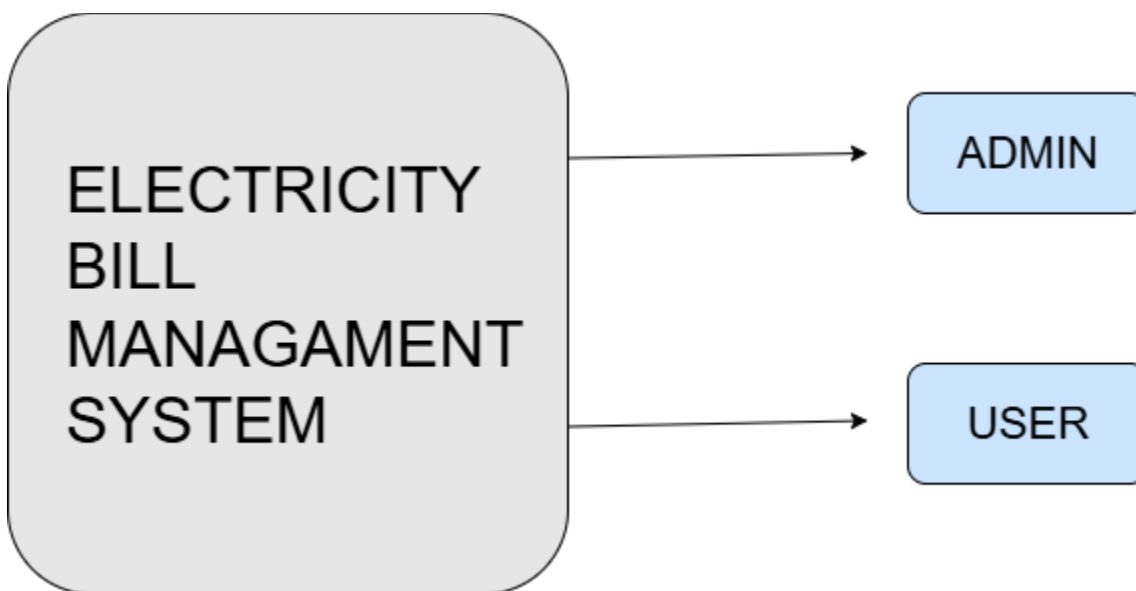
The User (Consumer) entity contains essential consumer details such as consumer_number, father_name, and contact_number, facilitating accurate billing and customer relationship management.

These entities are interconnected through fundamental relationships: Vendors are managed by Admins, Users are served by Vendors in their respective posting areas, and Admins oversee the entire billing lifecycle. This refined model ensures data integrity while supporting operational workflows like vendor assignment, consumer registration, and administrative oversight, providing a robust foundation for database implementation.

4.2.1.2 Data-Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a simple map that shows how information moves through a system. Instead of focusing on the technical details, it answers the basic questions: **Where does data come from?** (like a customer submitting a meter reading), **What processes does it go through?** (like calculating the bill amount), and **Where does it end up?** (like a bill being generated or a payment being recorded). It uses easy-to-understand symbols: circles for processes, arrows for data flow, rectangles for external people or systems, and open-ended rectangles for data storage. For our Electricity Bill System, a DFD helps us visually map out the entire journey of data, from a customer's initial registration to the final bill payment, ensuring we don't miss any important steps before we start coding.

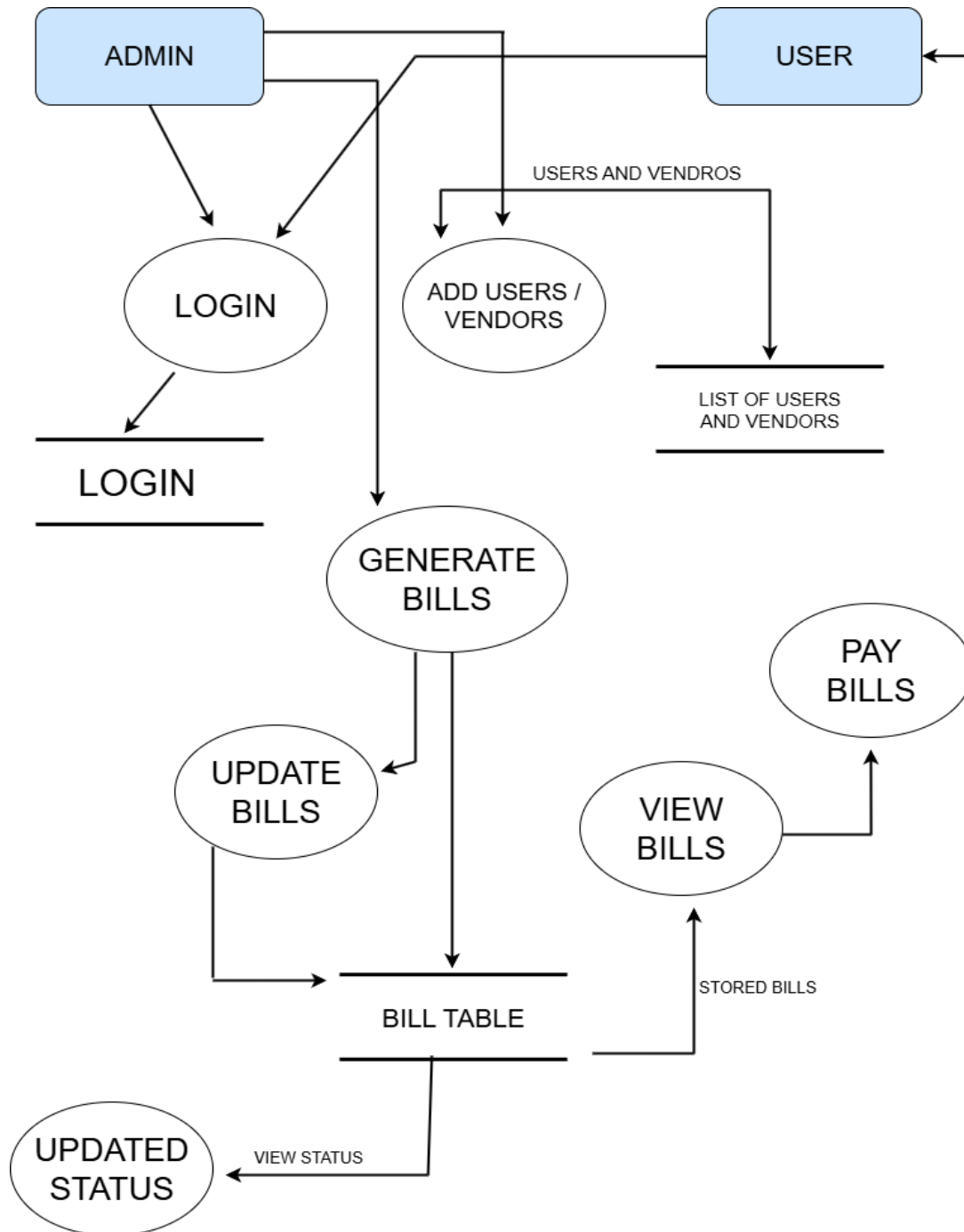
Level 0.0 DFD:



The Level 0 Data Flow Diagram of the Electricity Bill Management System outlines two external users: the **Admin** who oversees system management and reporting, also handles meter reading and bill generation, and

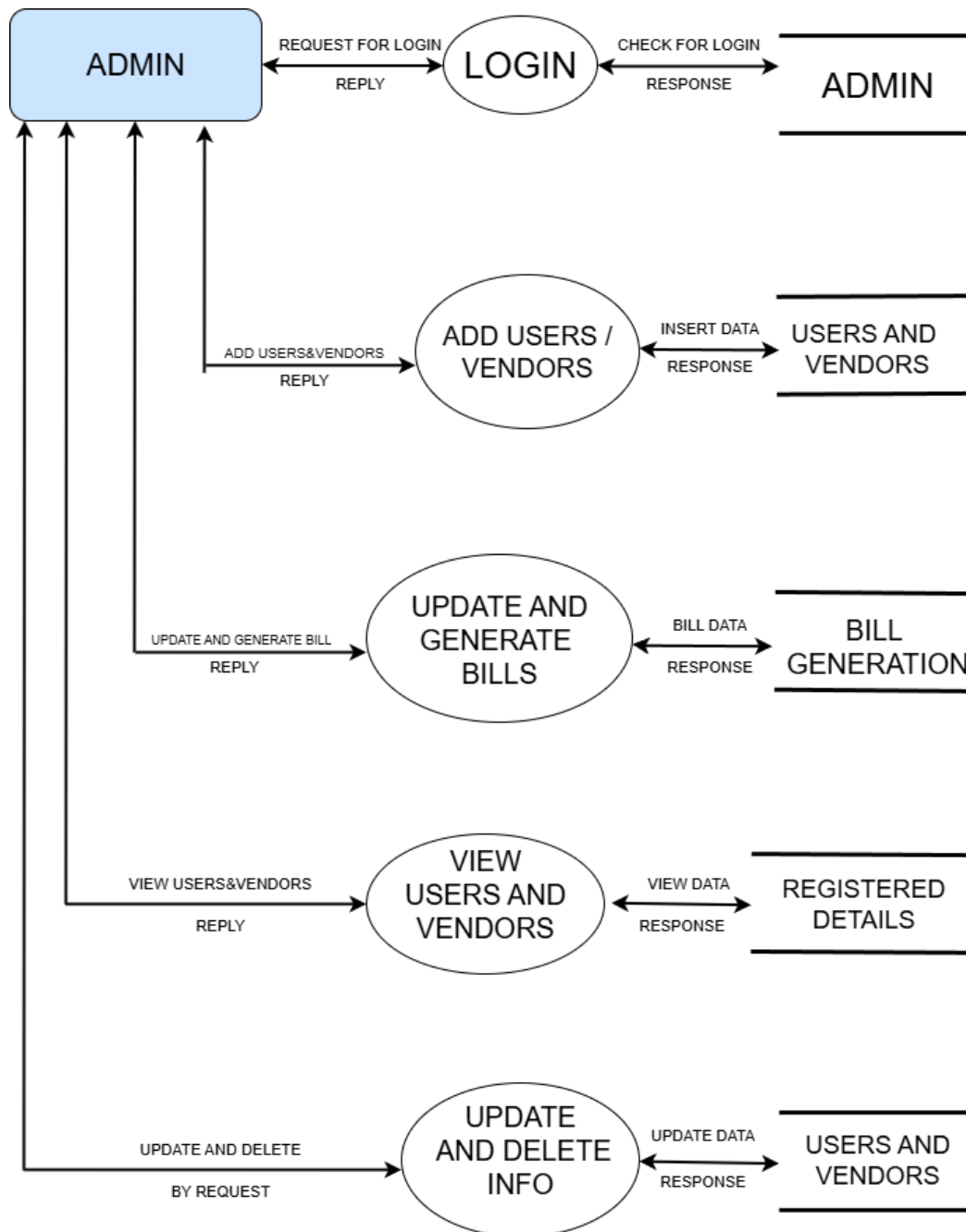
the **User** who views bills and makes payments. This diagram establishes the system's scope by showing how each user group interacts with the EBMS at the highest level.

Level 1.0 DFD:



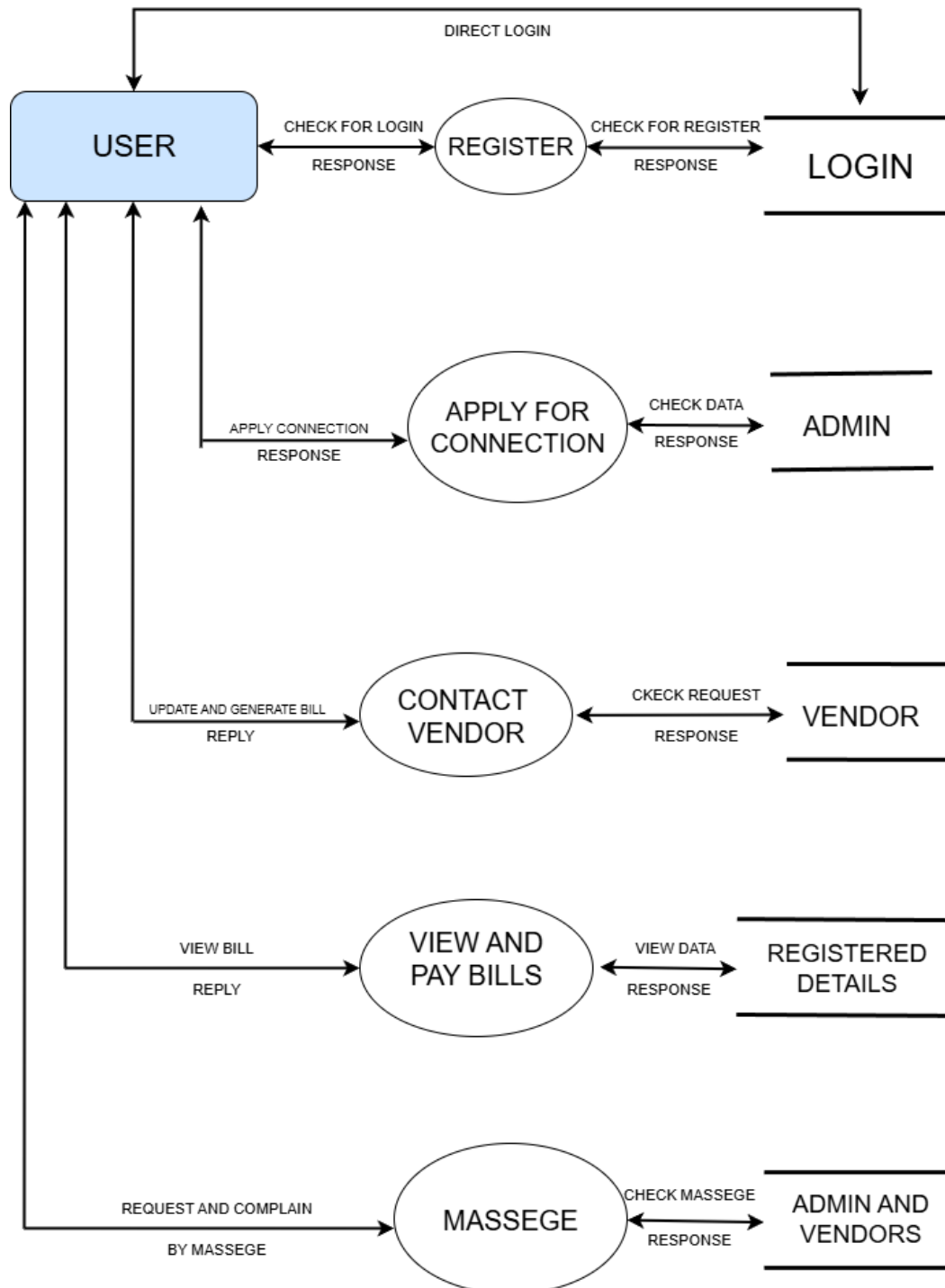
Level 1 Data Flow Diagram breaks down the internal processes of the EBMS. It details how **Login Authentication** leads to separate admin and user workflows. Admins manage **User/Vendor Data** and **Bill Generation**, while Users access **Bill Viewing** and **Payment Processing**. The diagram shows interactions with key data stores and illustrates how both user types can check payment status ('Paid'/'Unpaid'), providing clear picture of the system's internal data movement and processing logic.

Level 1.0.1 DFD:



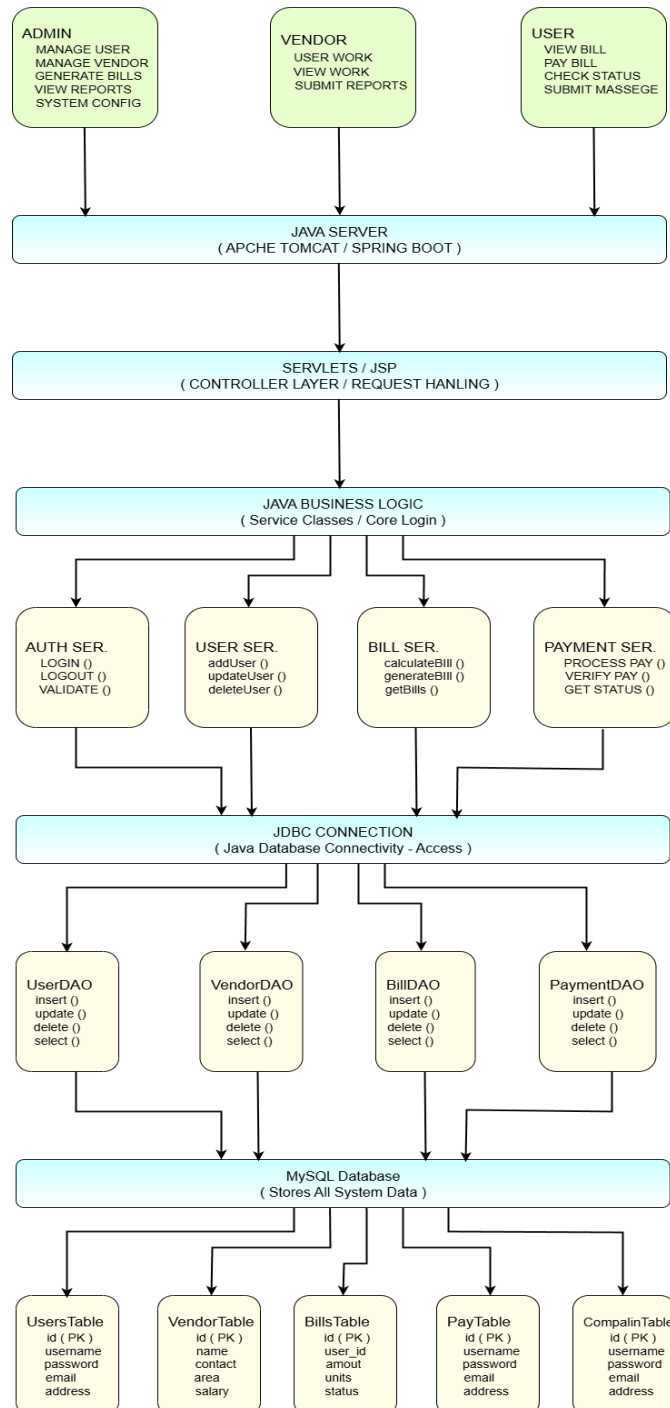
Admin Functions drills down into the specific sub-processes available to an administrator. The workflow starts with **Login Validation**, followed by a suite of management options: **Add/View Users & Vendors**, **Generate/Update Bills**, and **Handle Data Updates/Deletions** based on user requests and complaints. This diagram effectively maps the detailed data flow and system interactions required for complete administrative control over the EBMS.

Level 1.0.2 DFD:



User Functions elaborates on the complete cycle of user interactions. It starts with **Login/Registration**, followed by core services: **Applying for Connections**, **Contacting Vendors**, and **Viewing/Paying Bills**. The process concludes with a dedicated communication channel for **Messages**, **Complaints**, **Feedback**, and **Help Requests**. This diagram showcases the user's end-to-end experience and the various support mechanisms within the EBMS.

4.2.1.3 System Architecture Diagram (ASCII Version)



Application Tier (Java Specific):

1. **Servlets/JSP** - Handle HTTP requests and responses
2. **Service Classes** - Contain business logic (calculations, validations)
3. **DAO Classes** - Handle database operations (CRUD)
4. **JDBC** - Connects Java to MySQL database

Data Flow:

1. **Browser** → HTTP Request → **Servlet**
2. **Servlet** → calls → **Service Class**
3. **Service Class** → calls → **DAO Class**
4. **DAO Class** → JDBC → **MySQL Database**
5. **Response** travels back in reverse order

System Architecture (Java + MySQL Stack)

The Electricity Bill Management System implements a classic 3-Tier Architecture using Java EE technologies. The **Presentation Tier** consists of web interfaces (JSP/HTML) serving three user roles: Admin, Vendor, and Consumer. The **Application Tier** is built entirely in Java, utilizing Servlets for request handling and business logic encapsulated in dedicated service classes (UserService, BillService, PaymentService). Data persistence is managed through DAO (Data Access Object) classes using JDBC for MySQL connectivity. The **Data Tier** employs MySQL database storing all system data in normalized tables. This architecture ensures clear separation between user interface, business logic, and data storage, while leveraging Java's robustness for enterprise-level billing operations and MySQL's reliability for data integrity.

4.3 Technology Stack & Implementation (Coding)

4.3.1 Technology Stack Specification

The Electricity Bill Management System employs a robust and scalable technology stack designed to ensure reliability, security, and maintainability. The selection of technologies is based on industry standards, compatibility requirements, and the specific needs of a billing management system.

4.3.1.1 Backend Technologies

Java SE 8, Java Servlets, JDBC, Apache Tomcat 9.x

- **Java SE 8** serves as the core programming language, chosen for its platform independence, strong memory management, and extensive ecosystem of libraries suitable for enterprise applications.

- **Java Servlets** handle HTTP request-response cycles, providing a robust framework for managing user sessions, authentication, and business logic processing.
- **JDBC (Java Database Connectivity)** facilitates seamless communication between the Java applications and MySQL database, offering reliable transaction management and connection pooling capabilities.
- **Apache Tomcat 9.x** functions as the web server and servlet container, selected for its lightweight footprint, excellent performance characteristics, and strong compatibility with Java EE specifications.

4.3.1.2 Database Management System

MySQL 8.0, MySQL Connector/J 8.0

- **MySQL 8.0** serves as the relational database management system, chosen for its ACID compliance, excellent performance with read-intensive operations, and strong data integrity features essential for financial transactions.
- **MySQL Connector/J 8.0** provides the official JDBC driver for MySQL, ensuring optimal compatibility and performance when connecting Java applications to the MySQL database.

4.3.1.3 Development Tools & Environment

Eclipse IDE 2023, MySQL Workbench 8.0, Git

- **Eclipse IDE 2023** offers comprehensive development features including code completion, debugging tools, and integrated server management for efficient Java web application development.
- **MySQL Workbench 8.0** provides visual database design, SQL development, and administration capabilities, streamlining database management throughout the development lifecycle.
- **Git** enables version control and collaborative development, ensuring code integrity and facilitating feature branching and release management.

4.3.2 Implementation (Coding)

This phase converts the system design into a functional application. The implementation uses Java Enterprise Edition, with Servlets handling the controller logic and JDBC for database connectivity. Development follows an incremental approach, starting with core authentication before moving to complex billing and payment features. Each module is continuously integrated and tested against the SRS requirements to ensure the final product meets all intended performance and behavior criteria.

Coding:

Import Declarations with Package

```
package [package name];
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.sql.Connection;
import java.sql.DriverManager;
```

```

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Calendar;
import java.util.Random;
import java.util.Scanner;

New Customer Registration Table

public void registerNewCustomerTable() {

    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);
        Statement s = c.createStatement();

        String q = "CREATE TABLE IF NOT EXISTS Register_New_Customer("
        + "Consumer_Number VARCHAR(20) UNIQUE, "
        + "Full_Name VARCHAR(50), "
        + "Father_Name VARCHAR (50), "
        + "Adhaar_Number VARCHAR (20) UNIQUE, "
        + "Contact_Number VARCHAR (15) UNIQUE, "
        + "Email VARCHAR (100), "
        + "Address VARCHAR (100), "
        + "State VARCHAR (30), "
        + "City VARCHAR (30), "
        + "District VARCHAR (30), "
        + "Zip_Code INT)";

        s.execute(q);
        System.out.println("Table Created Successfully");
        s.close();
        c.close();
    } catch (Exception e) {
        System.out.println("Exception Error in adminRegisterNewCustomerTable: " +
        e.getMessage());
        e.printStackTrace();
    }
}

```

New Customer Registration Data Insertion

```
public void registerNewCustomer() {
    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);

        String q = "INSERT INTO Register_New_Customer("
            + "Consumer_Number, "
            + "Full_Name, "
            + "Father_Name, "
            + "Adhaar_Number, "
            + "Contact_Number, "
            + "Email, "
            + "Address, "
            + "State, "
            + "City, "
            + "District, "
            + "Zip_Code) "
            + "VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

        PreparedStatement ps = c.prepareStatement(q);
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        @SuppressWarnings("resource")
        Scanner sc = new Scanner(System.in);

        System.out.print("Consumer Number (CN/CNAME/SNO): ");
        String Consumer_Number = br.readLine();
        ps.setString(1, Consumer_Number);

        System.out.print("Full name: ");
        String Full_Name = br.readLine();
        ps.setString(2, Full_Name);

        System.out.print("Father's Name: ");
        String Father_Name = br.readLine();
        ps.setString(3, Father_Name);
```

```

System.out.print("Adhaar Number: ");
String Adhaar_Number = br.readLine();
ps.setString(4, Adhaar_Number);

System.out.print("Contact Number: ");
String Contact_Number = br.readLine();
ps.setString(5, Contact_Number);

System.out.print("E-mail: ");
String Email = br.readLine();
ps.setString(6, Email);

System.out.print("Address: ");
String Address = br.readLine();
ps.setString(7, Address);

System.out.print("State: ");
String State = br.readLine();
ps.setString(8, State);

System.out.print("City: ");
String City = br.readLine();
ps.setString(9, City);

System.out.print("District: ");
String District = br.readLine();
ps.setString(10, District);

System.out.print("Zip Code: ");
int Zip_Code = Integer.parseInt(br.readLine());
ps.setInt(11, Zip_Code);

Random rand = new Random();
System.out.print("Captcha Code: ");
int sum = 0;

for (int i = 0; i < 3; i++) {
    int random = rand.nextInt(10);
    System.out.print(random);
    if (i < 2) {

```

```

System.out.print(" + ");
}
sum += random;
}

System.out.println("\nEnter the Captcha Code: ");
int cc = sc.nextInt();

if (sum == cc) {
ps.executeUpdate();
System.out.println("Captcha Code Matched");
System.out.println("\nCustomer Registered Successfully!");
} else {
System.out.println("Captcha Code Not Matched");
System.out.println("No Data Inserted");
}
c.close();
} catch (Exception e) {
System.out.println("Exception Error in adminRegisterNewCustomerData: " +
e.getMessage());
e.printStackTrace();
}
}

```

View Customers which are registered

```

public void viewCustomers() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);
System.out.println("1. View customer by Consumer Number");
System.out.println("2. View All Customers");
System.out.print("Enter your choice : ");
int choice = sc.nextInt();

```

```

if (choice == 1) {
    System.out.print("Enter Customer Consumer Number : ");
    String consumerNumber = sc.next();

    String cq = "SELECT * FROM Register_New_Customer WHERE Consumer_Number = ?";
    PreparedStatement ps = c.prepareStatement(cq);
    ps.setString(1, consumerNumber);
    ResultSet rs = ps.executeQuery();

    while (rs.next()) {
        int random = 0, i, cc, sum = 0;
        Random rand = new Random();
        System.out.print("Captcha Code: ");

        for (i = 0; i < 3; i++) {
            random = rand.nextInt(10);
            System.out.print(random);
            if (i < 2) {
                System.out.print(" + ");
            }
            sum += random;
        }

        System.out.println("");
        System.out.print("Enter the Captcha Code = ");
        cc = sc.nextInt();

        if (sum == cc) {
            System.out.println("Captcha code matched");
            System.out.println("\n===== Registered Customers =====");
            System.out.println("Consumer Number: " + rs.getString("Consumer_Number"));
            System.out.println("Full Name: " + rs.getString("Full_Name"));
            System.out.println("Father's Name: " + rs.getString("Father_Name"));
            System.out.println("Aadhaar: " + rs.getLong("Adhaar_Number"));
            System.out.println("Contact: " + rs.getString("Contact_Number"));
            System.out.println("Email: " + rs.getString("Email"));
            System.out.println("Address: " + rs.getString("Address") + ", " +
                rs.getString("City") + ", " + rs.getString("District") + " - " +
                rs.getInt("Zip_Code") + ", " + rs.getString("State"));
            System.out.println("-----");
        } else {

```

```

System.out.println("Captcha not matched.");
}
}
} else if (choice == 2) {
String cq = "SELECT * FROM Register_New_Customer";
PreparedStatement ps = c.prepareStatement(cq);
ResultSet rs = ps.executeQuery();
        int random = 0, i, cc, sum = 0;

Random rand = new Random();
System.out.print("Captcha Code: ");

for (i = 0; i < 3; i++) {
random = rand.nextInt(10);
System.out.print(random);
if (i < 2) {
System.out.print(" + ");
}
sum += random;
}

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();

if (sum == cc) {
System.out.println("Captcha code matched");
System.out.println("\n===== Registered Customers =====");

while (rs.next()) {
System.out.println("Consumer Number: " + rs.getString("Consumer_Number"));
System.out.println("Full Name: " + rs.getString("Full_Name"));
System.out.println("Father's Name: " + rs.getString("Father_Name"));
System.out.println("Aadhaar: " + rs.getLong("Adhaar_Number"));
System.out.println("Contact: " + rs.getString("Contact_Number"));
System.out.println("Email: " + rs.getString("Email"));
System.out.println("Address: " + rs.getString("Address") + ", " +
rs.getString("City") + ", " + rs.getString("District") + " - " +
rs.getInt("Zip_Code") + ", " + rs.getString("State"));
System.out.println("-----");
}
} else {

```

```

System.out.println("Captcha not matched.");
}
} else {
System.out.println("Invalid Choice!");
}

c.close();
} catch (Exception e) {
System.out.println("Exception Error in displayAllCustomers " + e.getMessage());
}
}

```

Customer Bill Generation Table

```

public void generateCustomerBillTable() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

Statement createTable = c.createStatement();
String q = "CREATE TABLE IF NOT EXISTS Customer_Bill ("
            + "ID VARCHAR(20),"
            + "Consumer_Number VARCHAR(12),"
            + "Customer_Name VARCHAR(30), "
            + "Units_Consumed INT, "
            + "Rate_Per_Unit INT, "
            + "Total_Amount INT, "
            + "Payable_Amount INT, "
            + "Status VARCHAR(20) DEFAULT 'Pending' " + ")";

createTable.execute(q);
System.out.println("Table Created for Bill Generation");
c.close();
} catch (Exception e) {
System.out.println("Exception Error in adminGenerateCustomerBillTable: " +
e.getMessage());
}
}

```

Customer Bill Generation Data Insertion

```
@SuppressWarnings("resource")
public void generateCustomerBill() {
    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Customer Consumer_Number : ");
        String Consumer_Number = sc.nextLine();

        String cq = "SELECT Consumer_Number, Full_Name, Father_Name, Contact_Number " +
            "FROM Register_New_Customer WHERE Consumer_Number = ?";
        PreparedStatement ps = c.prepareStatement(cq);
        ps.setString(1, Consumer_Number);
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            String consumerNumber = rs.getString("Consumer_Number");
            String customerName = rs.getString("Full_Name");
            String fatherName = rs.getString("Father_Name");
            String contactNumber = rs.getString("Contact_Number");

            System.out.println("\nCustomer Found!");
            System.out.println("Consumer Number: " + consumerNumber);
            System.out.println("Customer Name: " + customerName);
            System.out.println("Father's Name: " + fatherName);
            System.out.println("Mobile Number: " + contactNumber);
            System.out.println("-----");

            System.out.print("Enter Rate Per Unit: ");
            int ratePerUnit = sc.nextInt();

            System.out.print("Enter Current month (1-12) : ");
            int currentMonth = sc.nextInt();

            String monthShort = switch (currentMonth) {
```

```

case 1 -> "JAN";
case 2 -> "FEB";
case 3 -> "MAR";
case 4 -> "APR";
case 5 -> "MAY";
case 6 -> "JUN";
case 7 -> "JUL";
case 8 -> "AUG";
case 9 -> "SEP";
case 10 -> "OCT";
case 11 -> "NOV";
case 12 -> "DEC";
default -> "UNK";
};
System.out.println("Month : " + monthShort);
String consumerSerial =
Consumer_Number.substring(Consumer_Number.lastIndexOf("/") + 1);

String cq2 = "SELECT Units_Consumed FROM Customer_Bill WHERE ID LIKE ? ORDER BY
ID DESC LIMIT 1";
PreparedStatement ps2 = c.prepareStatement(cq2);
ps2.setString(1, "/" + consumerSerial);
ResultSet rs2 = ps2.executeQuery();

int prevUnits = 0;
if (rs2.next()) {
prevUnits = rs2.getInt("Units_Consumed");
}
rs2.close();
ps2.close();

System.out.println("Units Consumed in Previous Month : " + prevUnits);
System.out.print("Enter units consumed this month: ");
int thisMonthUnits = sc.nextInt();
int currentConsumed = thisMonthUnits - prevUnits;
if (currentConsumed < 0)
currentConsumed = thisMonthUnits;

System.out.println("Total Units consumed this month: " + currentConsumed);

int totalAmount = currentConsumed * ratePerUnit;

```

```

int discountAmount = 0;
String areaType = "";

if (ratePerUnit == 8) {
    areaType = "Rural Area";
    if (totalAmount >= 10000)
        discountAmount = (totalAmount * 10) / 100;
    else if (totalAmount >= 5000)
        discountAmount = (totalAmount * 5) / 100;
    } else if (ratePerUnit == 10) {
        areaType = "Urban Area";
        if (totalAmount >= 12000)
            discountAmount = (totalAmount * 10) / 100;
        else if (totalAmount >= 10000)
            discountAmount = (totalAmount * 5) / 100;
        } else if (ratePerUnit == 12) {
            areaType = "Commercial Area";
            if (totalAmount >= 15000)
                discountAmount = (totalAmount * 10) / 100;
            else if (totalAmount >= 12000)
                discountAmount = (totalAmount * 5) / 100;
            } else {
                System.out.println("Invalid rate per unit entered!");
                return;
            }

int payableAmount = totalAmount - discountAmount;
System.out.println("\n=== BILL DETAILS ===");
System.out.println("Area Type: " + areaType);
System.out.println("Total Amount: ₹" + totalAmount);
if (discountAmount > 0) {
    System.out.println("Discount Amount: ₹" + discountAmount);
} else {
    System.out.println("No Discount Applied");
}
System.out.println("Payable Amount: ₹" + payableAmount);

int year = Calendar.getInstance().get(Calendar.YEAR) % 100;
String newID = monthShort + "/" + year + "/" + consumerSerial;
System.out.println("Generated Bill ID: " + newID);

```

```

Random rand = new Random();
System.out.print("\nCaptcha Code: ");
int sum = 0;
for (int i = 0; i < 3; i++) {
    int random = rand.nextInt(10);
    System.out.print(random);
    if (i < 2)
        System.out.print(" + ");
    sum += random;
}

System.out.print("\nEnter the Captcha Code: ");
int cc = sc.nextInt();

if (sum == cc) {

    String insertQuery = "INSERT INTO Customer_Bill " + "(ID, Consumer_Number,
    Customer_Name, Units_Consumed, Rate_Per_Unit, Total_Amount, Payable_Amount,
    Status) " + "VALUES(?, ?, ?, ?, ?, ?, ?, 'Pending')";
    PreparedStatement pss = c.prepareStatement(insertQuery);
    pss.setString(1, newID);
    pss.setString(2, consumerNumber);
    pss.setString(3, customerName);
    pss.setInt(4, currentConsumed);
    pss.setInt(5, ratePerUnit);
    pss.setInt(6, totalAmount);
    pss.setInt(7, payableAmount);

    pss.executeUpdate();
    System.out.println("Bill generated successfully!");
    pss.close();
} else {
    System.out.println("Captcha verification failed! Bill not generated.");
}

} else {
    System.out.println("Customer ID not found in database!");
}
rs.close();
ps.close();

```

```

c.close();

} catch (Exception e) {
System.out.println("Exception Error in generateCustomerBill: " + e.getMessage());
}
}

```

View Bills Generated, Check Status, and view Complaints

```

public void viewBills() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

@SuppressWarnings("resource")
Scanner sc1 = new Scanner(System.in);
System.out.println("1. Display Customer's bill by ID");
System.out.println("2. Display All Customer bills");
System.out.println("3. View Bill Status");
System.out.println("4. View Bill Complains");
System.out.print("Enter your choice : ");
int choice = sc1.nextInt();
if (choice == 1) {
@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);
System.out.print("Enter Customer ID: ");
String ID = sc.next();

String cq = "SELECT * FROM Customer_Bill WHERE ID = ?";
PreparedStatement ps1 = c.prepareStatement(cq);
ps1.setString(1, ID);
ResultSet rs1 = ps1.executeQuery();
String fullName = "", fatherName = "";
long contactNo = 0;

while (rs1.next()) {
String consumerNumber = rs1.getString("Consumer_Number");

```

```

String cqr = "SELECT Full_Name, Father_Name, Contact_Number FROM
register_new_customer WHERE Consumer_Number = ?";
PreparedStatement ps2 = c.prepareStatement(cqr);
ps2.setString(1, consumerNumber);
ResultSet rs2 = ps2.executeQuery();

if (rs2.next()) {
fullName = rs2.getString("Full_Name");
fatherName = rs2.getString("Father_Name");
contactNo = rs2.getLong("Contact_Number");
}
int random = 0, i, cc, sum = 0;
Random rand = new Random();
System.out.print("Captcha Code: ");

for (i = 0; i < 3; i++) {
random = rand.nextInt(10);
System.out.print(random);
if (i < 2) {
System.out.print(" + ");
}
sum += random;
}

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();
if (sum == cc) {
System.out.println("Captcha code matched \n");
System.out.println("===== Customer Bills =====");
System.out.println("Customer ID: " + rs1.getString("ID"));
System.out.println("Customer Name: " + fullName);
System.out.println("Father's Name: " + fatherName);
System.out.println("Units Consumed: " + rs1.getInt("Units_Consumed"));
System.out.println("Rate Per Unit: " + rs1.getInt("Rate_Per_Unit"));
System.out.println("Total Amount: ₹ " + rs1.getInt("Total_Amount"));
System.out.println("Payable Amount: ₹ " + rs1.getInt("Payable_Amount"));
System.out.println("Contact Number: " + contactNo);

System.out.println("-----");
}

```

```

}
} else if (choice == 2) {
@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);
String cq = "SELECT * FROM Customer_Bill";
PreparedStatement ps1 = c.prepareStatement(cq);
ResultSet rs1 = ps1.executeQuery();

int random = 0, i, cc, sum = 0;
Random rand = new Random();
System.out.print("Captcha Code: ");

for (i = 0; i < 3; i++) {
random = rand.nextInt(10);
System.out.print(random);
if (i < 2) {
System.out.print(" + ");
}
sum += random;
}

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();
if (sum == cc) {
System.out.println("Captcha code matched \n");
System.out.println("===== Customer Bills =====");

while (rs1.next()) {
String customerId = rs1.getString("ID");
String cqr = "SELECT Full_Name, Father_Name, Contact_Number " + "FROM
register_new_customer WHERE ID = ?";
PreparedStatement ps2 = c.prepareStatement(cqr);
ps2.setString(1, customerId);
ResultSet rs2 = ps2.executeQuery();
String fullName = "", fatherName = "";
long contactNo = 0;
if (rs2.next()) {
fullName = rs2.getString("Full_Name");
fatherName = rs2.getString("Father_Name");
contactNo = rs2.getLong("Contact_Number");

```

```

}

System.out.println("Customer ID: " + customerId);
System.out.println("Customer Name: " + fullName);
System.out.println("Father's Name: " + fatherName);
System.out.println("Units Consumed: " + rs1.getInt("Units_Consumed"));
System.out.println("Rate Per Unit: " + rs1.getInt("Rate_Per_Unit"));
System.out.println("Total Amount: ₹ " + rs1.getInt("Total_Amount"));
System.out.println("Payable Amount: ₹ " + rs1.getInt("Payable_Amount"));
System.out.println("Contact Number: " + contactNo);
System.out.println("-----");
}
} else {
System.out.println("Captcha not matched.");
}

} else if (choice == 3) {
@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);
System.out.print("Enter Customer ID: ");
String ID = sc.next();

String cq = "SELECT billID, Customer_Name, Payable_Amount, Status FROM
Customer_Bill WHERE Consumer_Number = ?";
PreparedStatement ps1 = c.prepareStatement(cq);
ps1.setString(1, ID);
ResultSet rs1 = ps1.executeQuery();

while (rs1.next()) {
int random = 0, i, cc, sum = 0;
Random rand = new Random();
System.out.print("Captcha Code: ");

for (i = 0; i < 3; i++) {
random = rand.nextInt(10);
System.out.print(random);
if (i < 2) {
System.out.print(" + ");
}
sum += random;
}
}
}

```

```

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();

if (sum == cc) {
    System.out.println("Captcha code matched \n");
    System.out.println("===== Customer's Bill Status =====");
    System.out.println("Customer ID: " + rs1.getString("ID"));
    System.out.println("Name: " + rs1.getString("Customer_Name"));
    System.out.println("Payable Amount: ₹ " + rs1.getInt("Payable_Amount"));
    System.out.println("Status: " + rs1.getString("Status"));
    System.out.println("-----");
} else {
    System.out.println("Captcha code not matched");
}
}
} else if (choice == 4) {
    @SuppressWarnings("resource")
    Scanner sc = new Scanner(System.in);
    System.out.println("1. View Bill Complain by ID");
    System.out.println("2. View All Complains");
    System.out.print("Choose an Option : ");
    int choose = sc.nextInt();

    if (choose == 1) {
        System.out.print("Enter Customer Consumer Number: ");
        String Consumer_Number = sc.next();
        String cq = "SELECT * FROM complaints WHERE Consumer_Number = ?";
        PreparedStatement ps1 = c.prepareStatement(cq);
        ps1.setString(1, Consumer_Number);
        ResultSet rs1 = ps1.executeQuery();
        while (rs1.next()) {
            int random = 0, i, cc, sum = 0;
            Random rand = new Random();
            System.out.print("Captcha Code: ");
            for (i = 0; i < 3; i++) {
                random = rand.nextInt(10);
                System.out.print(random);
                if (i < 2) {
                    System.out.print(" + ");
                }
            }
        }
    }
}

```

```

}
sum += random;
}

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();

if (sum == cc) {
    System.out.println("Captcha code matched \n");
    System.out.println("===== Customer's Complain =====");
    System.out.println("Customer Consumer Number: " +
rs1.getString("Consumer_Number"));
    System.out.println("Name: " + rs1.getString("Customer_Name"));
    System.out.println("About Complaint: " + rs1.getString("About"));
    System.out.println("Bill Complaint: " + rs1.getString("Complaint"));
    System.out.println("-----");
}
}
} else if (choose == 2) {
    String cq = "SELECT * FROM complaints";
    PreparedStatement ps1 = c.prepareStatement(cq);
    ResultSet rs1 = ps1.executeQuery();
    while (rs1.next()) {
        int random = 0, i, cc, sum = 0;
        Random rand = new Random();
        System.out.print("Captcha Code: ");

        for (i = 0; i < 3; i++) {
            random = rand.nextInt(10);
            System.out.print(random);
            if (i < 2) {
                System.out.print(" + ");
            }
            sum += random;
        }

        System.out.println("");
        System.out.print("Enter the Captcha Code = ");
        cc = sc.nextInt();
    }
}

```

```

if (sum == cc) {
    System.out.println("Captcha code matched \n");
    System.out.println("===== Customer's Complain =====");
    System.out.println("Customer Consumer Number: " +
rs1.getString("Consumer_Number"));
    System.out.println("Name: " + rs1.getString("Customer_Name"));
    System.out.println("About Complaint: " + rs1.getString("About"));
    System.out.println("Bill Complaint: " + rs1.getString("Complaint"));
    System.out.println("-----");
}
}
} else {
    System.out.println("Incorrect Option Chosen!");
}
} else {
    System.out.println("Invalid Choice!");
}
c.close();

} catch (Exception e) {
    System.out.println("Exception Error in viewBills " + e.getMessage());
}
}

```

Vendor Registration Table

```

public void registerVendorTable() {
    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);

        Statement createTable = c.createStatement();
        String q = "CREATE TABLE IF NOT EXISTS register_New_Vendor ("
+ "Vendor_ID VARCHAR(30) PRIMARY KEY, "
+ "Name VARCHAR (50) NOT NULL, "
+ "Father_Name VARCHAR(50), "
+ "Mother_Name VARCHAR(50), "
+ "Qualification VARCHAR(50), "
+ "Experience INT, "
+ "Contact_Number VARCHAR(10), "

```

```

+ "Email VARCHAR(50), "
+ "Address VARCHAR(50), "
+ "State VARCHAR(50), "
+ "City VARCHAR(50), "
+ "District VARCHAR(50), "
+ "Zip_Code INT, "
+ "Posting_Area VARCHAR(50), "
+ "Job_Post VARCHAR(50), "
+ "Salary DECIMAL(10,2)" + ")";
createTable.execute(q);
System.out.println("Table Created for Vendor Registration");
c.close();
} catch (Exception e) {
System.out.println("Exception Error in registerVendorTable: " + e.getMessage());
}
}

```

Vendor Registration Data Insertion

```

public void registerNewVendor() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

String q = "INSERT INTO Register_New_Vendor(Vendor_ID, Name, Father_Name,
Mother_Name, Qualification, Experience, Contact_Number, Email, Address, State,
City, District, Zip_Code, Posting_Area, Job_Post,
Salary)VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

PreparedStatement ps = c.prepareStatement(q);
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
Scanner sc = new Scanner(System.in);

System.out.print("Vendor ID: ");
String vID = br.readLine();
ps.setString(1, vID);

System.out.print("Full name: ");

```

```
String Full_Name = br.readLine();
ps.setString(2, Full_Name);

System.out.print("Father's Name: ");
String Father_Name = br.readLine();
ps.setString(3, Father_Name);

System.out.print("Mother Name: ");
String Mother_Name = br.readLine();
ps.setString(4, Mother_Name);

System.out.print("Qualifications: ");
String Qualification = br.readLine();
ps.setString(5, Qualification);

System.out.print("Experience: ");
int Experience = Integer.parseInt(br.readLine());
ps.setInt(6, Experience);

System.out.print("Contact Number: ");
String Contact_Number = br.readLine();
ps.setString(7, Contact_Number);

System.out.print("E-mail ID: ");
String Email = br.readLine();
ps.setString(8, Email);

System.out.print("Address: ");
String Address = br.readLine();
ps.setString(9, Address);

System.out.print("State: ");
String State = br.readLine();
ps.setString(10, State);

System.out.print("City: ");
String City = br.readLine();
ps.setString(11, City);

System.out.print("District: ");
String District = br.readLine();
```

```

ps.setString(12, District);

System.out.print("Zip Code: ");
int Zip_Code = Integer.parseInt(br.readLine());
ps.setInt(13, Zip_Code);

System.out.print("Posting Area: ");
String Posting_Area = br.readLine();
ps.setString(14, Posting_Area);

System.out.print("Job Post: ");
String Job_Post = br.readLine();
ps.setString(15, Job_Post);

System.out.print("Salary: ");
double Salary = Double.parseDouble(br.readLine());
ps.setDouble(16, Salary);

Random rand = new Random();
System.out.print("Captcha Code: ");
int sum = 0;

for (int i = 0; i < 3; i++) {
    int random = rand.nextInt(10);
    System.out.print(random);
    if (i < 2) {
        System.out.print(" + ");
    }
    sum += random;
}

System.out.println("\nEnter the Captcha Code: ");
int cc = sc.nextInt();

if (sum == cc) {
    ps.executeUpdate();
    System.out.println("Captcha Code Matched");
    System.out.println("\nVendor Registered Successfully!");
    System.out.println("-----");
    System.out.println("Full Name: " + Full_Name);
    System.out.println("Father's Name: " + Father_Name);
}

```

```

System.out.println("Mother's Name: " + Mother_Name);
System.out.println("Qualifications: " + Qualification);
System.out.println("Experience: " + Experience + " years");
System.out.println("Mobile Number: " + Contact_Number);
System.out.println("E-mail ID: " + Email);
System.out.format("Full Address: %s, %s, %s %d, %s", Address, City, District,
Zip_Code, State);
System.out.println("\nPosting Area: " + Posting_Area);
System.out.println("Job Post: " + Job_Post);
System.out.println("Salary: " + Salary);
System.out.println("Data Inserted Successfully!");
} else {
System.out.println("Captcha Code Not Matched");
System.out.println("No Data Inserted");
}
sc.close();
br.close();
ps.close();
c.close();
} catch (Exception e) {
System.out.println("Exception Error in registerNewVendor: " + e.getMessage());
}
}

```

View Vendors

```

public void viewVendor() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);
System.out.println("1. View vendor by Vendor_ID");
System.out.println("2. View All Vendors");
System.out.print("Enter your choice : ");
int choice = sc.nextInt();

if (choice == 1) {

```

```

System.out.print("Enter Vendor ID : ");
String vID = sc.next();

String cq = "SELECT * FROM Register_New_Vendor WHERE Vendor_ID = ?";
PreparedStatement ps = c.prepareStatement(cq);
ps.setString(1, vID);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    int random = 0, i, cc, sum = 0;
    Random rand = new Random();
    System.out.print("Captcha Code: ");

    for (i = 0; i < 3; i++) {
        random = rand.nextInt(10);
        System.out.print(random);
        if (i < 2) {
            System.out.print(" + ");
        }
        sum += random;
    }

    System.out.println("");
    System.out.print("Enter the Captcha Code = ");
    cc = sc.nextInt();

    if (sum == cc) {
        System.out.println("Captcha code matched");
        System.out.println("\n===== Registered Vendor by ID =====");
        System.out.println("Vendor ID: " + rs.getString("Vendor_ID"));
        System.out.println("Full Name: " + rs.getString("Name"));
        System.out.println("Father's Name: " + rs.getString("Father_Name"));
        System.out.println("Mother's Name: " + rs.getString("Mother_Name"));
        System.out.println("Qualifications: " + rs.getString("Qualification"));
        System.out.println("Experience: " + rs.getInt("Experience"));
        System.out.println("Mobile Number: " + rs.getString("Contact_Number"));
        System.out.println("Email ID: " + rs.getString("Email"));
        System.out.println("Full Address: " + rs.getString("Address") + ", " +
            rs.getString("City") + ", " + rs.getString("District") + " - " +
            rs.getInt("Zip_Code") + ", " + rs.getString("State"));
        System.out.println("Posting Area: " + rs.getString("Posting_Area"));
        System.out.println("Job Post: " + rs.getString("Job_Post"));
    }
}

```

```

System.out.println("Salary: ₹ " + rs.getDouble("Salary"));
System.out.println("-----");
} else {
System.out.println("Captcha not matched.");
}
}
} else if (choice == 2) {
String cq = "SELECT * FROM Register_New_Vendor";
PreparedStatement ps = c.prepareStatement(cq);
ResultSet rs = ps.executeQuery();
int random = 0, i, cc, sum = 0;
Random rand = new Random();
System.out.print("Captcha Code: ");

for (i = 0; i < 3; i++) {
random = rand.nextInt(10);
System.out.print(random);
if (i < 2) {
System.out.print(" + ");
}
sum += random;
}

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();

if (sum == cc) {
System.out.println("Captcha code matched");
System.out.println("\n===== Registered All Vendors =====");

while (rs.next()) {
System.out.println("Vendor ID: " + rs.getString("Vendor_ID"));
System.out.println("Full Name: " + rs.getString("Name"));
System.out.println("Father's Name: " + rs.getString("Father_Name"));
System.out.println("Mother's Name: " + rs.getString("Mother_Name"));
System.out.println("Qualifications: " + rs.getString("Qualification"));
System.out.println("Experience: " + rs.getInt("Experience"));
System.out.println("Mobile Number: " + rs.getString("Contact_Number"));
System.out.println("Email_ID: " + rs.getString("Email"));
}
}
}

```

```

System.out.println("Full Address: " + rs.getString("Address") + ", " +
rs.getString("City") + ", " + rs.getString("District") + " - " +
rs.getInt("Zip_Code") + ", " + rs.getString("State"));
System.out.println("Posting Area: " + rs.getString("Posting_Area"));
System.out.println("Job Post: " + rs.getString("Job_Post"));
System.out.println("Salary: ₹ " + rs.getDouble("Salary"));

                System.out.println("-----");
    }
} else {
System.out.println("Captcha not matched.");
}
}

c.close();
} catch (Exception e) {
System.out.println("Exception Error in viewVendors " + e.getMessage());
}
}

```

Data Deletion

```

@SuppressWarnings("resource")
public void deleteData() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);
Statement smt = c.createStatement();
String q = "delete from [Table_Name] where ID = ' '";

Scanner sc = new Scanner(System.in);
int random = 0, i, cc, sum = 0;
Random rand = new Random();
System.out.print("Captcha Code: ");

for (i = 0; i < 3; i++) {
random = rand.nextInt(10);
System.out.print(random);
if (i < 2) {

```

```

System.out.print(" + ");
}
sum += random;
}

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();

if (sum == cc) {
    System.out.println("Captcha code matched");
    smt.execute(q);
    System.out.println("Deleted");
} else {
    System.out.println("Captcha code not matched");
}
} catch (Exception e) {
    System.out.println("Exception Error in DeleteData" + e.getMessage());
}
}

```

Update Data

```

public void updateData() {
    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);
        Statement smt = c.createStatement();
        String q = "UPDATE Register_New_Customer SET Address = ' ' WHERE Adhaar_Number = ";
        @SuppressWarnings("resource")
        Scanner sc = new Scanner(System.in);
        int random = 0, i, cc, sum = 0;
        Random rand = new Random();
        System.out.print("Captcha Code: ");

        for (i = 0; i < 3; i++) {
            random = rand.nextInt(10);
            System.out.print(random);
            if (i < 2) {

```

```

System.out.print(" + ");
}
sum += random;
}

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();

if (sum == cc) {
System.out.println("Captcha code matched");
smt.execute(q);
System.out.println("Updated");
} else {
System.out.println("Captcha code not matched");
}
} catch (Exception e) {
System.out.println("Exception Error in updateData" + e.getMessage());
}
}

```

User Registration Table

```

public void userRegistrationTable() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";

Connection c = DriverManager.getConnection(url, name, pass);
Statement s = c.createStatement();
String q = "CREATE TABLE IF NOT EXISTS user_Registration("
+ "Name varchar(30), "
+ "Email varchar(50), "
+ "Password varchar(50), "
+ "Contact_Number varchar(12), "
+ "Consumer_Number varchar(15))";
s.execute(q);
System.out.println("Table Created for User Registration");
c.close();
s.close();
}
}

```

```

} catch (Exception e) {
System.out.println("Exception error in userRegistrationTable " + e.getMessage());
}
}

```

User Registration Data Insertion

```

public void userRegistration() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);
@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);
String q = "insert into user_Registration(Name, Email, Password, Contact_Number,
Consumer_Number) values(?,?,?,?,?)";
PreparedStatement ps = c.prepareStatement(q);
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

System.out.print("Enter your Name: ");
String userName = br.readLine();
ps.setString(1, userName);

System.out.print("Enter Email_ID: ");
String emailID = br.readLine();
ps.setString(2, emailID);

System.out.print("Enter Password: ");
String userPass = br.readLine();
ps.setString(3, userPass);

System.out.print("Enter Mobile Number: ");
String phoneNumber = br.readLine();
ps.setString(4, phoneNumber);

System.out.print("Enter Consumer Number (CN/CNAME/SNO): ");
String consumerNumber = br.readLine();
ps.setString(5, consumerNumber);

int random = 0, i, cc, sum = 0;

```

```

Random rand = new Random();

System.out.print("Captcha Code: ");
for (i = 0; i < 3; i++) {
    random = rand.nextInt(10);
    System.out.print(random);
    if (i < 2) {
        System.out.print(" + ");
    }
    sum += random;
}

System.out.println("");
System.out.print("Enter the Captcha Code = ");
cc = sc.nextInt();

if (sum == cc) {
    ps.executeUpdate();
    System.out.println("Captcha code Matched\nSuccessfully Registered");
} else {
    System.out.println("Captcha Code Not Matched\nRegistration Failed");
}

ps.close();
c.close();
} catch (Exception e) {
    System.out.println("Exception error in userRegistration " + e.getMessage());
}
}

```

User Login which are already Registered

```

@SuppressWarnings("resource")
public void login() {
    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter User ID: ");
    }
}

```

```

String UserID = sc.nextLine();
System.out.print("Enter User Password: ");
String UPassword = sc.nextLine();
String q = "SELECT * FROM user_Registration where Email='" + UserID + "'And
Password='" + UPassword + "'";
Statement st = c.createStatement();
ResultSet rs = st.executeQuery(q);

if (rs.next()) {
String uemail = rs.getString("Email");
String upass = rs.getString("Password");

if (UserID.equals(uemail) && UPassword.equals(upass)) {
System.out.println("Login Successfully!.... Welcome " + rs.getString("Name"));

while (true) {
System.out.println("\n----- User Dashboard -----");
System.out.println("1. Apply for New Connection");
System.out.println("2. Contact to Vendor");
System.out.println("3. View Bill");
System.out.println("4. Pay Bill");
System.out.println("5. Message");
System.out.println("6. Logout");
System.out.println("-----");
System.out.print("Choose an option: ");
int choice2 = sc.nextInt();
switch (choice2) {
case 1:
applyConnection();
break;
case 2:
vendorContact();
break;
case 3:
viewBill();
break;
case 4:
payBill();
break;
case 5:

```

```

Message();
break;
case 6:
System.out.println("Logging out...");
break;
default:
System.out.println("Invalid choice. Please try again.");
}
}
} else {
System.out.println("Invalid User ID and Password");
}
} else {
System.out.println("Invalid User ID and Password");
}
}
st.execute(q);
c.close();
} catch (Exception e) {
System.out.println("Exception error in login " + e.getMessage());
}
}

```

Apply for Connection Table

```

public void applyConnectionTable() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

String q = "CREATE TABLE IF NOT EXISTS Apply_for_Connection(Upload_House_Reciept
VARCHAR(20) DEFAULT 'Uploaded')";

Statement stmt = c.createStatement();
stmt.executeUpdate(q);
System.out.println("Apply_for_Connection table created successfully!");

stmt.close();
c.close();
} catch (Exception e) {

```

```

System.out.println("Exception Error in applyConnectionTable: " + e.getMessage());
}
}

```

Apply for Connection (user)

```

public void applyConnection() {
    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);

        @SuppressWarnings("resource")
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Request ID : ");
        String rID = sc.next();

        String cq = "SELECT Consumer_Number, Request_Type FROM requests WHERE Request_Id = ?";
        PreparedStatement ps = c.prepareStatement(cq);
        ps.setString(1, rID);
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            String cn = rs.getString("Consumer_Number");
            String message = rs.getString("Request_Type");

            String q2 = "SELECT Full_Name, Address FROM Register_New_Customer WHERE Consumer_Number = ?";
            PreparedStatement ps2 = c.prepareStatement(q2);
            ps2.setString(1, cn);
            ResultSet rs2 = ps2.executeQuery();

            if (rs2.next()) {
                String n = rs2.getString("Full_Name");
                String add = rs2.getString("Address");

                int sum = 0;
                Random rand = new Random();
                System.out.print("Captcha Code: ");
            }
        }
    }
}

```

```

for (int i = 0; i < 3; i++) {
    int random = rand.nextInt(10);
    System.out.print(random);
    if (i < 2)
        System.out.print(" + ");
    sum += random;
}
System.out.println();

System.out.print("Enter the Captcha Code = ");
int cc = sc.nextInt();

if (cc == sum) {
    String q = "INSERT INTO Apply_for_Connection(Upload_House_Reciept) VALUES (?)";
    PreparedStatement ps3 = c.prepareStatement(q);
    ps3.setString(1, "Uploaded");
    ps3.executeUpdate();

    System.out.println("Application submitted successfully!");
    System.out.println("Name : " + n);
    System.out.println("Consumer Number : " + cn);
    System.out.println("Address : " + add);
    System.out.println("Message : " + message);

    ps3.close();
} else {
    System.out.println("Captcha incorrect. Please try again.");
}
rs2.close();
ps2.close();
} else {
    System.out.println("No request found with this Request ID.");
}

rs.close();
ps.close();
c.close();

} catch (Exception e) {
    System.out.println("Exception error in applyConnection: " + e.getMessage());
}

```

```
}  
}
```

Vendor Contact (user)

```
public void vendorContact() {  
    try {  
        String url = "jdbc:mysql://localhost:3306/[database name]";  
        String name = "[username]";  
        String pass = "[password]";  
        Connection c = DriverManager.getConnection(url, name, pass);  
  
        @SuppressWarnings("resource")  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter Vendor's name : ");  
        String vName = sc.next();  
  
        String q = "SELECT Posting_Area, Contact_Number from Register_New_Vendor where  
Name = ?";  
        PreparedStatement ps = c.prepareStatement(q);  
        ps.setString(1, vName);  
        ResultSet rs = ps.executeQuery();  
  
        if (rs.next()) {  
            String contactnumber = rs.getString("Contact_Number");  
            String postingarea = rs.getString("Posting_Area");  
  
            int sum = 0;  
            Random rand = new Random();  
            System.out.print("Captcha Code: ");  
            for (int i = 0; i < 3; i++) {  
                int random = rand.nextInt(10);  
                System.out.print(random);  
                if (i < 2)  
                    System.out.print(" + ");  
                sum += random;  
            }  
            System.out.println();  
  
            System.out.print("Enter the Captcha Code = ");  
            int cc = sc.nextInt();
```

```

if (cc == sum) {

System.out.println("Vendor Contact Information");
System.out.println("Mobile Number : " + contactnumber);
System.out.println("Posting Area : " + postingarea);
ps.close();
} else {
System.out.println("Captcha incorrect. Please try again.");
}
}

} catch (Exception e) {
System.out.println("Exception error in vendorContact " + e.getMessage());
}

}

```

View Bill (user)

```

public void viewBill() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);
@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);

System.out.println("1. View Consumption Details");
System.out.println("2. View Bill Payment History");
System.out.print("Choose an option : ");
int opt = sc.nextInt();
if (opt == 1) {
System.out.print("\nEnter Consumer Number: ");
String consumerNumber = sc.nextLine();

String q = "SELECT * FROM Register_New_Customer WHERE Consumer_Number = ?";
PreparedStatement psConsumer = c.prepareStatement(q);
psConsumer.setString(1, consumerNumber);
ResultSet rs = psConsumer.executeQuery();

```

```

if (rs.next()) {
String cNo = rs.getString("Consumer_Number");
String cName = rs.getString("Full_Name");

String q2 = "SELECT * FROM customer_Bill WHERE Consumer_Number = ? ORDER BY ID
DESC LIMIT 1";
PreparedStatement psLast = c.prepareStatement(q2);
psLast.setString(1, cNo);
ResultSet rsLast = psLast.executeQuery();

if (rsLast.next()) {
System.out.println("-----Bill for Current Month-----\n");
System.out.println("Customer's Consumer Number : " + cNo);
System.out.println("Name: " + cName);
System.out.println("Month of Billing: " + rsLast.getString("Month"));
System.out.println("Units Consumed in Current Month: " +
rsLast.getInt("Units_Consumed"));
System.out.println("Rate per Unit: ₹" + rsLast.getDouble("Rate_Per_Unit"));
System.out.println("Total Amount: ₹" + rsLast.getDouble("Total_Amount"));
System.out.println("Status: " + rsLast.getString("Status"));
} else {
System.out.println("No bill found for this Consumer Number");
}
} else {
System.out.println("Consumer not Found");
}
} else if (opt == 2) {
System.out.print("\nEnter Consumer Number : ");
String consumer = sc.nextLine();

String q = "SELECT * FROM Register_New_Customer WHERE Consumer_Number = ?";
PreparedStatement psConsumer = c.prepareStatement(q);
psConsumer.setString(1, consumer);
ResultSet rs = psConsumer.executeQuery();

if (rs.next()) {
String cNo = rs.getString("Consumer_Number");
String cName = rs.getString("Full_Name");

```

```

String lastBillQuery = "SELECT * FROM Customer_Bill WHERE Consumer_Number = ?
ORDER BY Bill_Id DESC LIMIT 1";
PreparedStatement psLast = c.prepareStatement(lastBillQuery);
psLast.setString(1, cNo);
ResultSet rsLast = psLast.executeQuery();

if (rsLast.next()) {
System.out.println("\n----- Current Month Bill -----");
System.out.println("Consumer Number : " + cNo);
System.out.println("Name : " + cName);
System.out.println("Billing Month : " + rsLast.getString("CurrentMonth"));
System.out.println("Units Consumed: " + rsLast.getInt("Units_Consumed"));
System.out.println("Rate per Unit : ₹" + rsLast.getDouble("Per_Unit_Price"));
System.out.println("-----");
System.out.println("Bill Amount : ₹" + rsLast.getDouble("Total_Amount"));
System.out.println("Status : " + rsLast.getString("Status"));
System.out.println("-----");
} else {
System.out.println("No bill found for this Consumer Number");
}
}
c.close();
}
} catch (Exception e) {
System.out.println("Exception error in viewBill " + e.getMessage());
}
}
}

```

Pay Bill (user)

```

@SuppressWarnings("resource")
public void payBill() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);
Scanner sc = new Scanner(System.in);
System.out.print("Enter Customer's ID : ");
String ID = sc.nextLine();

```

```

final String atmNo = "CCV/6151";
final String upiIDf = "6191@ybl";

String cq = "SELECT ID, Payable_Amount, Status FROM Customer_Bill WHERE ID = ?";
PreparedStatement ps = c.prepareStatement(cq);
ps.setString(1, ID);
ResultSet rs = ps.executeQuery();
if (rs.next()) {
    int billAmount = rs.getInt("Payable_Amount");
    String status = rs.getString("Status");

    System.out.println("Amount : " + billAmount);
    System.out.println("Status : " + status);

    System.out.println("Payment Method");
    System.out.println("1. ATM-card");
    System.out.println("2. UPI");
    System.out.print("Choose option: ");
    int opt = sc.nextInt();

    if (opt == 1) {
        System.out.print("Enter ATM-card No (CCV/4 digit ATM) : ");
        String atmNo = sc.next();
        if (atmNo.equals(atmNo)) {
            System.out.print("Enter the Amount : ");
            int payAmount = sc.nextInt();
            if (payAmount == billAmount) {
                System.out.println("Payment Successful");
                updateBillStatus(ID);
            } else {
                System.out.println("Invalid Payment\nPayment Failed");
            }
        } else {
            System.out.println("Invalid ATM Pin\nPayment Failed");
        }
    } else if (opt == 2) {
        System.out.print("Enter the UPI_ID (00001@ybl) : ");
        String upiID = sc.next();
        if (upiID.equals(upiIDf)) {
            System.out.print("Enter the Amount : ");
            int payAmount = sc.nextInt();

```

```

if (payAmount == billAmount) {
    System.out.println("Payment Successful");
    updateBillStatus(ID);
} else {
    System.out.println("Invalid Payment\nPayment Failed");
}
} else {
    System.out.println("Invalid UPI Pin\nPayment Failed");
}
} else {
    System.out.println("Invalid payment option");
}
} else {
    System.out.println("No bill found for the given ID");
}

rs.close();
ps.close();
c.close();
} catch (Exception e) {
    System.out.println("Exception Error in pay bill : " + e.getMessage());
}
}

```

Update Status (automatically after payment)

```

public void updateBillStatus(String billID) {
    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);
        String updateQuery = "UPDATE Customer_Bill SET Status = 'Paid' WHERE ID = ?";
        PreparedStatement ps = c.prepareStatement(updateQuery);
        ps.setString(1, billID);
        int rowsUpdated = ps.executeUpdate();

        if (rowsUpdated > 0) {
            System.out.println("Bill status updated to Paid");
        } else {
            System.out.println("Failed to update bill status");
        }
    }
}

```

```

}

ps.close();
c.close();
} catch (Exception e) {
System.out.println("Exception Error in updateBillStatus: " + e.getMessage());
}
}

```

Messages (user)

```

public void Message() {
Scanner sc = new Scanner(System.in);

System.out.println("Messages Section");
System.out.println("1.Complaints");
System.out.println("2.FeedBacks");
System.out.println("3.Request");
System.out.println("4.Help");
System.out.print("Enter your choice : ");
int choice = sc.nextInt();
switch (choice) {
case 1 -> userComplaints();
case 2 -> userFeedbacks();
case 3 -> userRequests();
case 4 -> askForHelp();
}
sc.close();
}

```

Complaints Table

```

public void userComplaintsTable() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

String q = "CREATE TABLE IF NOT EXISTS complaints("
+ "Complain_Id VARCHAR(20) PRIMARY KEY, "
+ "Consumer_Number VARCHAR(50), "

```

```

+ "Complain_Type VARCHAR(50), "
+ "Status VARCHAR(20) DEFAULT 'Pending', "
+ "Date DATE)";

Statement stmt = c.createStatement();
stmt.executeUpdate(q);
System.out.println("Complaints table created successfully!");

stmt.close();
c.close();
} catch (Exception e) {
System.out.println("Exception Error in userComplaintsTable: " + e.getMessage());
}
}

```

Complaints Insertion (user)

```

public void userComplaints() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

String getLastId = "SELECT Complain_Id FROM complaints ORDER BY Complain_Id DESC
LIMIT 1";
Statement stmt = c.createStatement();
ResultSet rs = stmt.executeQuery(getLastId);

String newComplainId = "CMP001";

if (rs.next()) {
String lastId = rs.getString("Complain_Id");
String num = lastId.replace("CMP", "");
int idNumber = Integer.parseInt(num);
idNumber = idNumber + 1;
if (idNumber < 10) {
newComplainId = "CMP00" + idNumber;
} else if (idNumber < 100) {
newComplainId = "CMP0" + idNumber;
} else {

```

```

newComplainId = "CMP" + idNumber;
}
}

String q = "INSERT INTO complaints(Complain_Id, Consumer_Number, Complain_Type,
Status, Date)"
+ " VALUES (?, ?, ?, ?, ?, ?)";
PreparedStatement ps = c.prepareStatement(q);

@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);

ps.setString(1, newComplainId);

System.out.print("Enter Consumer Number : CN/SBPDCL/");
String num = sc.next();
String consumerNumber = "CN/SBPDCL/" + num;
ps.setString(2, consumerNumber);

System.out.println("Complaints : \n");
System.out.println(" 1. Meter Issue");
System.out.println(" 2. Wrong Bill");
System.out.println(" 3. Power Supply Issue");
System.out.println(" 4. High Bill");
System.out.println(" 5. Other\n");
System.out.print("Enter your choice: ");
int choice = sc.nextInt();
sc.nextLine();

String complainType;
switch (choice) {
case 1 -> complainType = "Meter Issue";
case 2 -> complainType = "Wrong Bill Provided";
case 3 -> complainType = "Power Supply Issues";
case 4 -> complainType = "Consumed High Bill";
case 5 -> {
System.out.println("Express your issue : ");
complainType = sc.nextLine();
}
default -> {
System.out.println("Invalid choice, defaulting to Other.");
}
}

```

```

complainType = "Other";
}
}
ps.setString(3, complainType);

ps.setString(4, "Pending");

java.sql.Date sqlDate = new java.sql.Date(System.currentTimeMillis());
ps.setDate(6, sqlDate);

ps.executeUpdate();
System.out.println("\nComplaint Registered Successfully!");
System.out.println("Complaint ID: " + newComplainId);
System.out.println("Status: Pending");
System.out.println("Date: " + sqlDate);

ps.close();
stmt.close();
c.close();
} catch (Exception e) {
System.out.println("Error in userComplaints: " + e.getMessage());
}
}

```

Feedback Table

```

public void userFeedbacksTable() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

String q = "CREATE TABLE IF NOT EXISTS feedbacks(" Feedback_Id VARCHAR(20)
PRIMARY KEY, Consumer_Number VARCHAR(50), Feedback_Type VARCHAR(50), Status
VARCHAR(20) DEFAULT 'Pending', " + "Date DATE)";

Statement stmt = c.createStatement();
stmt.executeUpdate(q);
System.out.println("Feedbacks table created successfully!");
}
}

```

```

stmt.close();
c.close();
} catch (Exception e) {
System.out.println("Exception Error in userFeedbacksTable: " + e.getMessage());
}
}

```

Feedback Insertion (user)

```

public void userFeedbacks() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

String getLastId = "SELECT Feedback_Id FROM feedbacks ORDER BY Feedback_Id DESC
LIMIT 1";
Statement stmt = c.createStatement();
ResultSet rs = stmt.executeQuery(getLastId);

String newFeedbackId = "FDB001";
if (rs.next()) {
String lastId = rs.getString("Feedback_Id");
String num = lastId.replace("FDB", "");
int idNumber = Integer.parseInt(num);
idNumber = idNumber + 1;
if (idNumber < 10) {
newFeedbackId = "FDB00" + idNumber;
} else if (idNumber < 100) {
newFeedbackId = "FDB0" + idNumber;
} else {
newFeedbackId = "FDB" + idNumber;
}
}
}

String q = "INSERT INTO feedbacks(Feedback_Id, Consumer_Number, Feedback_Type,
Status, Date) VALUES (?, ?, ?, ?, ?, ?)";
PreparedStatement ps = c.prepareStatement(q);

@SuppressWarnings("resource")

```

```

Scanner sc = new Scanner(System.in);

ps.setString(1, newFeedbackId);

System.out.print("Enter Consumer No.: CN/SBPDCL/");
String num = sc.next();
String consumerNo = "CN/SBPDCL/" + num;
ps.setString(2, consumerNo);

System.out.println("Feedback Type: \n");
System.out.println(" 1. Service Quality");
System.out.println(" 2. Staff Behaviour");
System.out.println(" 3. Billing Process");
System.out.println(" 4. Website/App Experience");
System.out.println(" 5. Other\n");

System.out.print("Enter your choice: ");
int choice = sc.nextInt();
sc.nextLine();

String feedbackType;
switch (choice) {
    case 1 -> feedbackType = "Service Quality";
    case 2 -> feedbackType = "Staff Behaviour";
    case 3 -> feedbackType = "Billing Process";
    case 4 -> feedbackType = "Website/App Experience";
    case 5 -> {
        System.out.println("Express your issue : ");
        feedbackType = sc.nextLine();
    }
    default -> {
        System.out.println("Invalid choice, defaulting to Other.");
        feedbackType = "Other";
    }
}
ps.setString(3, feedbackType);

ps.setString(4, "Pending");

java.sql.Date sqlDate = new java.sql.Date(System.currentTimeMillis());
ps.setDate(6, sqlDate);

```

```

ps.executeUpdate();
System.out.println("\nFeedback Registered Successfully!");
System.out.println("Feedback ID: " + newFeedbackId);
System.out.println("Status: Pending");
System.out.println("Date: " + sqlDate);

ps.close();
stmt.close();
c.close();
} catch (Exception e) {
System.out.println("Error in userFeedbacks: " + e.getMessage());
}
}

```

Request Table

```

public void userRequestsTable() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

String q = "CREATE TABLE IF NOT EXISTS requests("
+ "Request_Id VARCHAR(20) PRIMARY KEY, "
+ "Consumer_Number VARCHAR(50), "
+ "Request_Type VARCHAR(50), "
+ "Status VARCHAR(20) DEFAULT 'Pending', "
+ "Date DATE)";

Statement stmt = c.createStatement();
stmt.executeUpdate(q);
System.out.println("Requests table created successfully!");

stmt.close();
c.close();
} catch (Exception e) {
System.out.println("Exception Error in userRequestsTable: " + e.getMessage());
}
}

```

Request Insertion (user)

```
public void userRequests() {
    try {
        String url = "jdbc:mysql://localhost:3306/[database name]";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);

        String getLastId = "SELECT Request_Id FROM requests ORDER BY Request_Id DESC
LIMIT 1";
        Statement stmt = c.createStatement();
        ResultSet rs = stmt.executeQuery(getLastId);

        String newRequestId = "REQ001";
        if (rs.next()) {
            String lastId = rs.getString("Request_Id");
            String num = lastId.replace("-", "");
            int idNumber = Integer.parseInt(num);
            idNumber = idNumber + 1;
            if (idNumber < 10) {
                newRequestId = "REQ00" + idNumber;
            } else if (idNumber < 100) {
                newRequestId = "REQ0" + idNumber;
            } else {
                newRequestId = "REQ" + idNumber;
            }
        }

        String q = "INSERT INTO requests(Request_Id, Consumer_Number, Request_Type,
Status, Date) VALUES (?, ?, ?, ?, ?)";
        PreparedStatement ps = c.prepareStatement(q);

        @SuppressWarnings("resource")
        Scanner sc = new Scanner(System.in);

        ps.setString(1, newRequestId);

        System.out.print("Enter Consumer No.: CN/SBPDCL/");
        String num = sc.next();
```

```

String consumerNo = "CN/SBPDCL/" + num;
ps.setString(2, consumerNo);

System.out.println("Request Type: \n");
System.out.println(" 1. Bill Payment Discount");
System.out.println(" 2. Early Supply Connection");
System.out.println(" 3. Load Increase Request");
System.out.println(" 4. Name Change Request");
System.out.println(" 5. Address Update Request");
System.out.println(" 6. Other");
System.out.print("\nEnter your choice: ");
int choice = sc.nextInt();
sc.nextLine();

String requestType;
switch (choice) {
case 1 -> requestType = "Bill Payment Discount";
case 2 -> requestType = "Early Supply Connection";
case 3 -> requestType = "Load Increase Request";
case 4 -> requestType = "Name Change Request";
case 5 -> requestType = "Address Update Request";
case 6 -> {
System.out.println("Express your issue : ");
requestType = sc.nextLine();
}
default -> {
System.out.println("Invalid choice, defaulting to Other.");
requestType = "Other";
}
}
ps.setString(3, requestType);

ps.setString(4, "Pending");

java.sql.Date sqlDate = new java.sql.Date(System.currentTimeMillis());
ps.setDate(5, sqlDate);

ps.executeUpdate();
System.out.println("\nRequest Registered Successfully!");
System.out.println("Request ID: " + newRequestId);
System.out.println("Status: Pending");

```

```

System.out.println("Date: " + sqlDate);

ps.close();
stmt.close();
c.close();
} catch (Exception e) {
System.out.println("Error in userRequests: " + e.getMessage());
}
}

```

Ask for Help Table

```

public void askForHelpTable() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";
String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);
String q = "CREATE TABLE IF NOT EXISTS ask_for_help("
+ "Help_Id VARCHAR(20) PRIMARY KEY, "
+ "Consumer_Number VARCHAR(50), "
+ "Help_Type VARCHAR(50), "
+ "Status VARCHAR(20) DEFAULT 'Pending', "
+ "Date DATE)";

Statement stmt = c.createStatement();
stmt.executeUpdate(q);
System.out.println("Ask For Help table created successfully!");

stmt.close();
c.close();
} catch (Exception e) {
System.out.println("Exception Error in askForHelpTable: " + e.getMessage());
}
}

```

Ask for Help Insertion (user)

```

@SuppressWarnings("resource")
public void askForHelp() {
try {
String url = "jdbc:mysql://localhost:3306/[database name]";

```

```

String name = "[username]";
String pass = "[password]";
Connection c = DriverManager.getConnection(url, name, pass);

String getLastId = "SELECT Help_Id FROM ask_for_help ORDER BY Help_Id DESC LIMIT
1";
Statement stmt = c.createStatement();
ResultSet rs = stmt.executeQuery(getLastId);

String newHelpId = "HLP001";
if (rs.next()) {
String lastId = rs.getString("Help_Id");
String num = lastId.replace("", "");
int idNumber = Integer.parseInt(num);
idNumber = idNumber + 1;
if (idNumber < 10) {
newHelpId = "HLP00" + idNumber;
} else if (idNumber < 100) {
newHelpId = "HLP0" + idNumber;
} else {
newHelpId = "HLP" + idNumber;
}
}

String q = "INSERT INTO ask_for_help(Help_Id, Consumer_Number, Help_Type, Status,
Date) VALUES (?, ?, ?, ?, ?)";
PreparedStatement ps = c.prepareStatement(q);

Scanner sc = new Scanner(System.in);

ps.setString(1, newHelpId);

System.out.print("Enter Consumer No.: CN/SBPDCL/");
String numPart = sc.next();
String consumerNo = "CN/SBPDCL/" + numPart;
ps.setString(2, consumerNo);

System.out.println("Help Type: ");
System.out.println("=====");
System.out.println(" 1. Technical Support");
System.out.println(" 2. Billing Help");

```

```

System.out.println(" 3. Supply Related Help");
System.out.println(" 4. Online Services Help");
System.out.println(" 5. Other");
System.out.println("=====");
System.out.print("Enter your choice: ");
int choice = sc.nextInt();
sc.nextLine();

String helpType;
switch (choice) {
case 1 -> helpType = "Technical Support";
case 2 -> helpType = "Billing Help";
case 3 -> helpType = "Supply Related Help";
case 4 -> helpType = "Online Services Help";
case 5 -> {
System.out.println("Express your issue : ");
helpType = sc.nextLine();
}
default -> {
System.out.println("Invalid choice, defaulting to Other.");
helpType = "Other";
}
}
ps.setString(3, helpType);

ps.setString(4, "Pending");

java.sql.Date sqlDate = new java.sql.Date(System.currentTimeMillis());
ps.setDate(5, sqlDate);

ps.executeUpdate();
System.out.println("\nHelp Request Registered Successfully!");
System.out.println("Help ID: " + newHelpId);
System.out.println("Status: Pending");
System.out.println("Date: " + sqlDate);

ps.close();
stmt.close();
c.close();
} catch (Exception e) {
System.out.println("Error in askForHelp: " + e.getMessage());
}

```

```
}  
}
```

Whole Methods Implementations (Run)

```
package [package name];  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.Statement;  
import java.util.Scanner;  
  
public class mainMaster {  
  
    // admin Login  
    public static boolean adminLogin() {  
        @SuppressWarnings("resource")  
        final Scanner sc1 = new Scanner(System.in);  
        System.out.print("Enter Admin Username: ");  
        String username = sc1.nextLine();  
        System.out.print("Enter Admin Password: ");  
        String password = sc1.nextLine();  
        return username.equals("admin_BMS") && password.equals("mod@2005");  
    }  
  
    public static void main(String[] args) {  
        // Object instantiation from classes  
        EBMS e = new EBMS();  
        mainMaster m = new mainMaster();  
  
        // Connectivity and Database creation  
        m.connectivity();  
        m.dbEBMS();  
        e.registerNewCustomerTable();  
        e.generateCustomerBillTable();  
        e.registerVendorTable();  
        e.userRegistrationTable();  
        e.userComplaintsTable();  
        e.userFeedbacksTable();  
        e.applyConnectionTable();  
    }  
}
```

```

// Running the EBMS
@SuppressWarnings("resource")
Scanner sc = new Scanner(System.in);
System.out.println("Welcome to EBMS");

while (true) { // Main loop
    System.out.println("1. Admin ");
    System.out.println("2. User ");
    System.out.println("3. Exit ");
    System.out.print("Enter your choice: ");
    int choice = sc.nextInt();

    if (choice == 1) {
        // Admin section
        if (!adminLogin()) {
            System.out.println("Invalid login");
            continue; // back to main menu
        } else {
            System.out.println("\nLogin Successful");
            // Admin stays logged in until they choose to logout or exit
            boolean adminLoggedIn = true;
            while (adminLoggedIn) { // Admin loop - stays logged in
                System.out.println("\n--- ADMIN PANEL ---");
                System.out.println("1. Register New Customer");
                System.out.println("2. View Customers");
                System.out.println("3. Generate Customer's Bill");
                System.out.println("4. View Customer Bills");
                System.out.println("5. Register New Vendor");
                System.out.println("6. View Vendor");
                System.out.println("7. Delete Data");
                System.out.println("8. Update Data");
                System.out.println("9. Logout (Back to Main Menu)");
                System.out.print("Choose an option: ");
                int choice2 = sc.nextInt();
                switch (choice2) {
                    case 1:
                        e.registerNewCustomer();
                        break;
                    case 2:
                        e.viewCustomers();
                        break;

```

```

case 3:
e.generateCustomerBill();
break;
case 4:
e.viewBills();
break;
case 5:
e.registerNewVendor();
break;
case 6:
e.viewVendor();
break;
case 7:
e.deleteData();
break;
case 8:
e.updateData();
break;
case 9:
System.out.println("Logging out... Returning to main menu...");
adminLoggedIn = false; // Exit admin loop
break;
default:
System.out.println("Invalid choice. Please try again.");
}
// Optional: Add a pause so user can see the result
if (adminLoggedIn && choice2 != 9) {
System.out.println("\nPress Enter to continue...");
try {
System.in.read();
} catch (Exception ex) {}
}
}
}
else if (choice == 2) {
// User section
while (true) { // User loop
System.out.println("\n--- USER SECTION ---");
System.out.println("1. Register");
System.out.println("2. Login");
System.out.println("3. Back to Main Menu");

```

```

System.out.print("Choose an option: ");
int uOpt = sc.nextInt();
if (uOpt == 1) {
    e.userRegistration();
} else if (uOpt == 2) {
    e.login();
} else if (uOpt == 3) {
    System.out.println("Returning to main menu...");
    break; // Exit user loop
} else {
    System.out.println("Invalid choice. Please try again.");
}
}
} else if (choice == 3) {
    System.out.println("Exiting EBMS. Goodbye!");
    break; // Exit whole program
} else {
    System.out.println("Invalid choice. Please try again.");
}
}
}

```

Connectivity

```

public void connectivity() {
    try {
        String url = "jdbc:mysql://localhost:3306";
        String name = "[username]";
        String pass = "[password]";
        Connection c = DriverManager.getConnection(url, name, pass);
        System.out.println("Connected");
        c.close();
    } catch (Exception e) {
        System.out.println("Exception Error in connectivity" + e);
    }
}

```

Database Creation

```

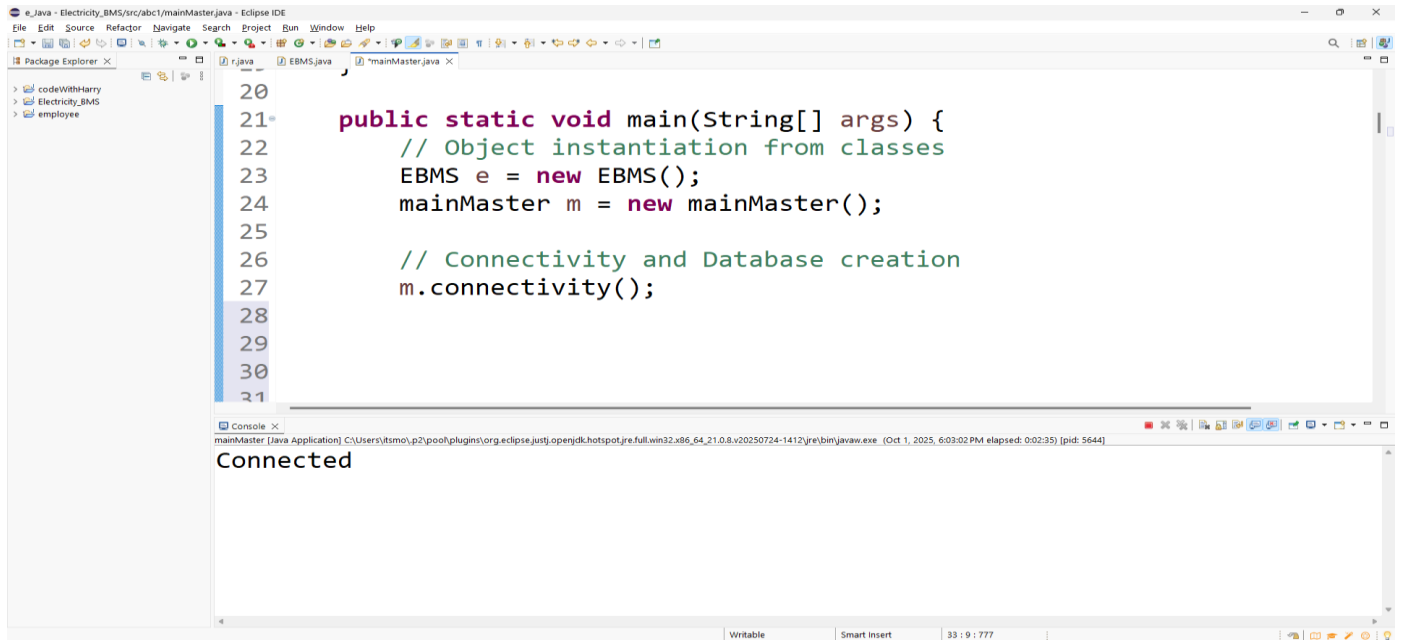
public void dbEBMS() {
    try {
        String url = "jdbc:mysql://localhost:3306";
        String name = "[username]";
        String pass = "[password]";
    }
}

```

```
Connection c = DriverManager.getConnection(url, name, pass);
Statement s = c.createStatement();
String q = "create database if not exists [database name]";
s.execute(q);
System.out.println("Database Created");
c.close();
} catch (Exception e) {
System.out.println("Exception Error in dbEBMS" + e);
}
}
}
```

4.4 Testing

Connectivity



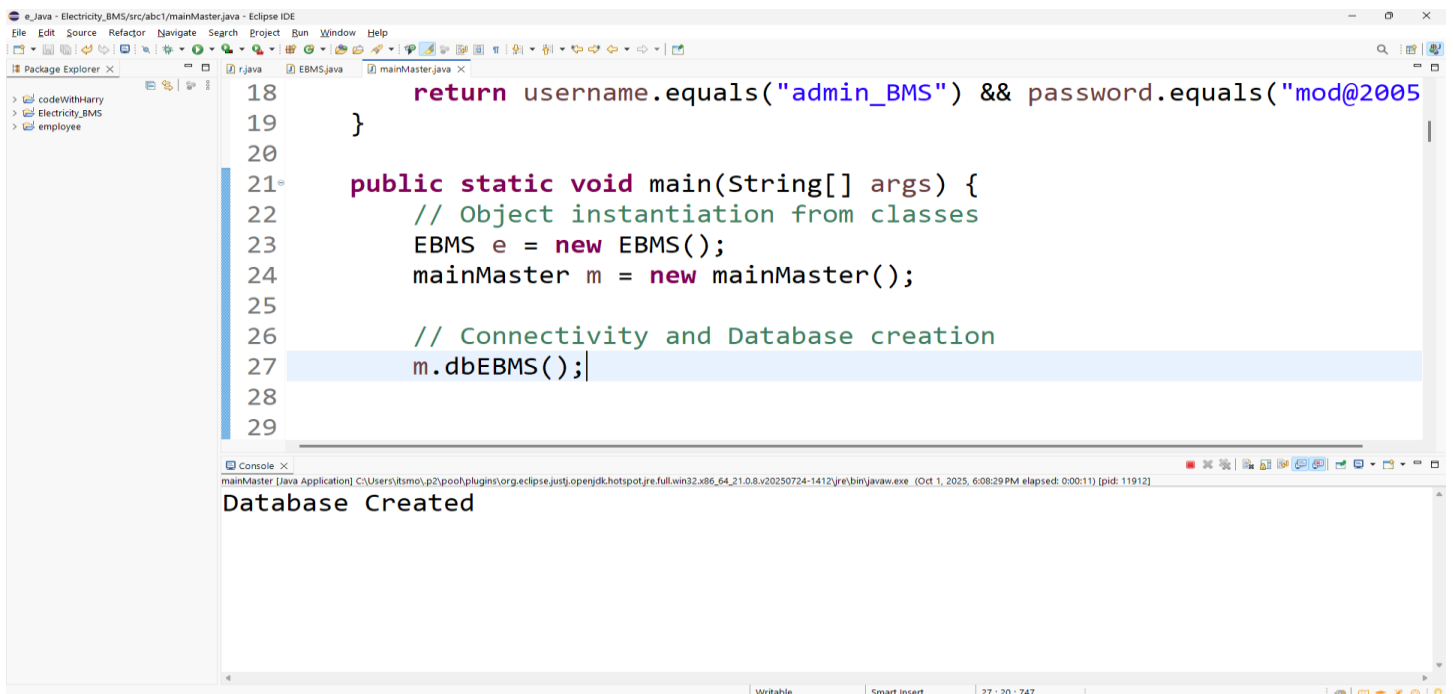
The screenshot shows the Eclipse IDE with the `mainMaster.java` file open. The code includes a `main` method that instantiates `EBMS` and `mainMaster` objects, and calls `m.connectivity()`. The console output shows the message "Connected".

```
20
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25
26     // Connectivity and Database creation
27     m.connectivity();
28
29
30
31
```

mainMaster (Java Application) C:\Users\itsmo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 1, 2025, 6:03:02 PM elapsed: 0:02:35) [pid: 5644]

Connected

Database Creation



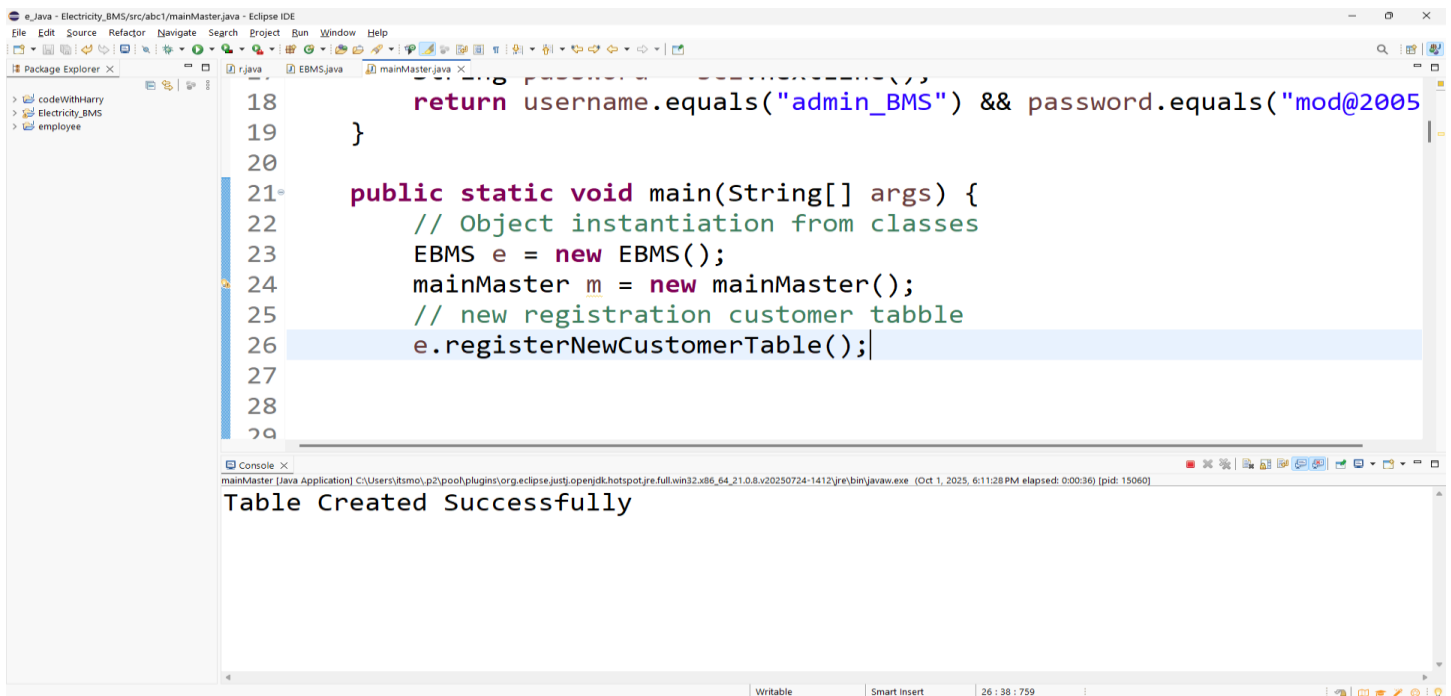
The screenshot shows the Eclipse IDE with the `mainMaster.java` file open. The code includes a `main` method that instantiates `EBMS` and `mainMaster` objects, and calls `m.dbEBMS()`. The console output shows the message "Database Created".

```
18     return username.equals("admin_BMS") && password.equals("mod@2005
19 }
20
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25
26     // Connectivity and Database creation
27     m.dbEBMS();
28
29
```

mainMaster (Java Application) C:\Users\itsmo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 1, 2025, 6:08:29 PM elapsed: 0:00:11) [pid: 11912]

Database Created

New Customer Registration Table

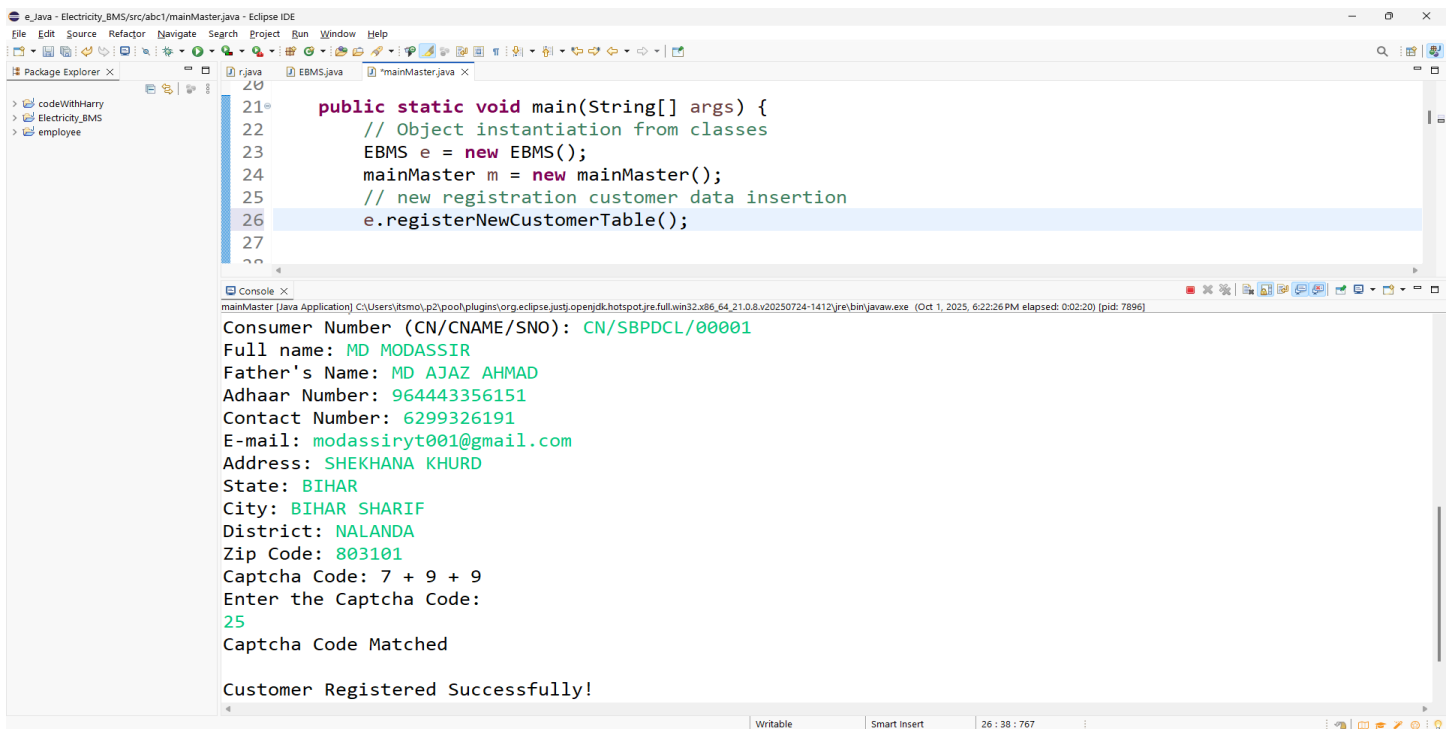


The screenshot shows the Eclipse IDE with the file `mainMaster.java` open. The code defines a `main` method that instantiates `EBMS` and `mainMaster` objects, then calls `e.registerNewCustomerTable();`. The console output shows "Table Created Successfully".

```
18         return username.equals("admin_BMS") && password.equals("mod@2005");
19     }
20
21     public static void main(String[] args) {
22         // Object instantiation from classes
23         EBMS e = new EBMS();
24         mainMaster m = new mainMaster();
25         // new registration customer table
26         e.registerNewCustomerTable();
27
28
29     }
```

Console Output: Table Created Successfully

New Customer Registration Data Insertion



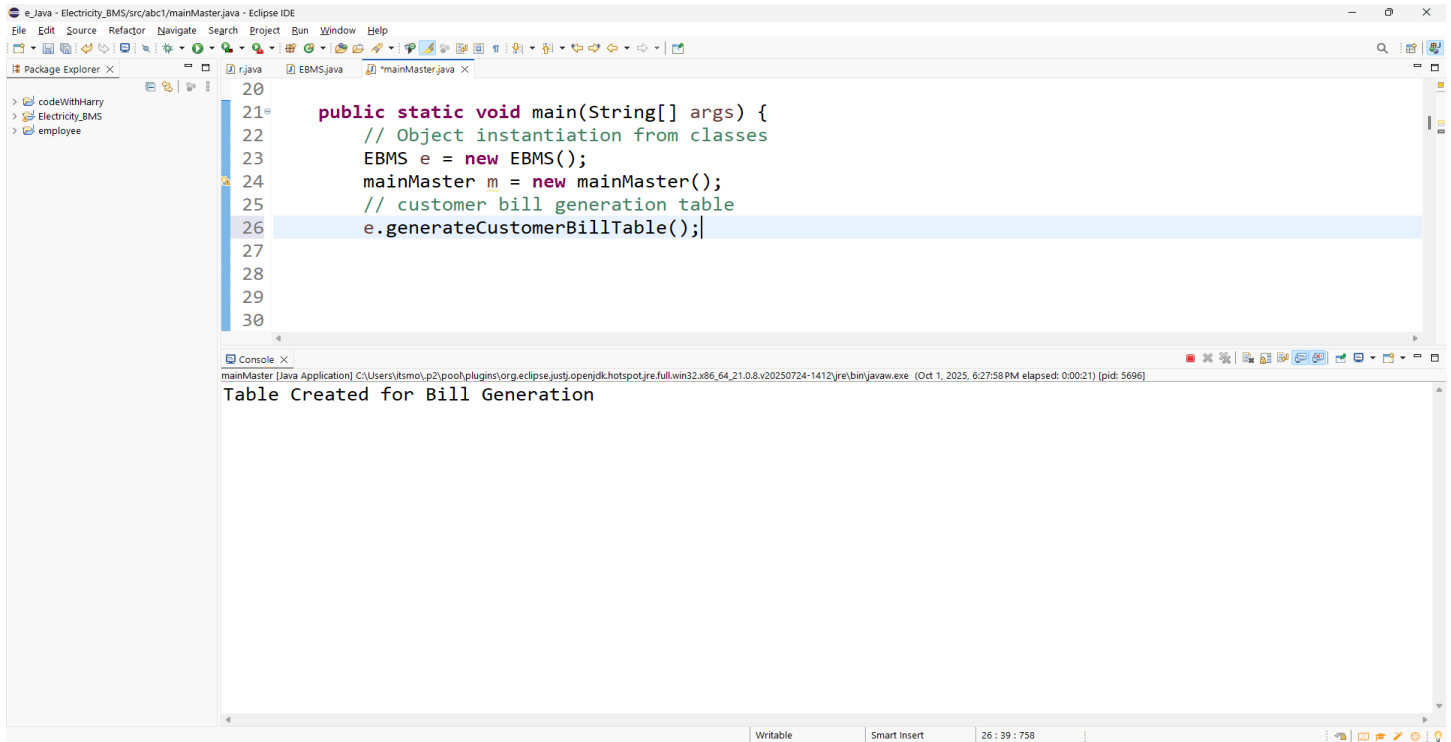
The screenshot shows the Eclipse IDE with the file `mainMaster.java` open. The code defines a `main` method that instantiates `EBMS` and `mainMaster` objects, then calls `e.registerNewCustomerTable();`. The console output shows the registration details for a new customer, including name, address, and contact information, and confirms successful registration.

```
21     public static void main(String[] args) {
22         // Object instantiation from classes
23         EBMS e = new EBMS();
24         mainMaster m = new mainMaster();
25         // new registration customer data insertion
26         e.registerNewCustomerTable();
27
28
29     }
```

Console Output:

```
Consumer Number (CN/CNAME/SNO): CN/SBPDCL/00001
Full name: MD MODASSIR
Father's Name: MD AJAZ AHMAD
Adhaar Number: 964443356151
Contact Number: 6299326191
E-mail: modassiry001@gmail.com
Address: SHEKHANA KHURD
State: BIHAR
City: BIHAR SHARIF
District: NALANDA
Zip Code: 803101
Captcha Code: 7 + 9 + 9
Enter the Captcha Code:
25
Captcha Code Matched
Customer Registered Successfully!
```

Customer Bill Generation Table



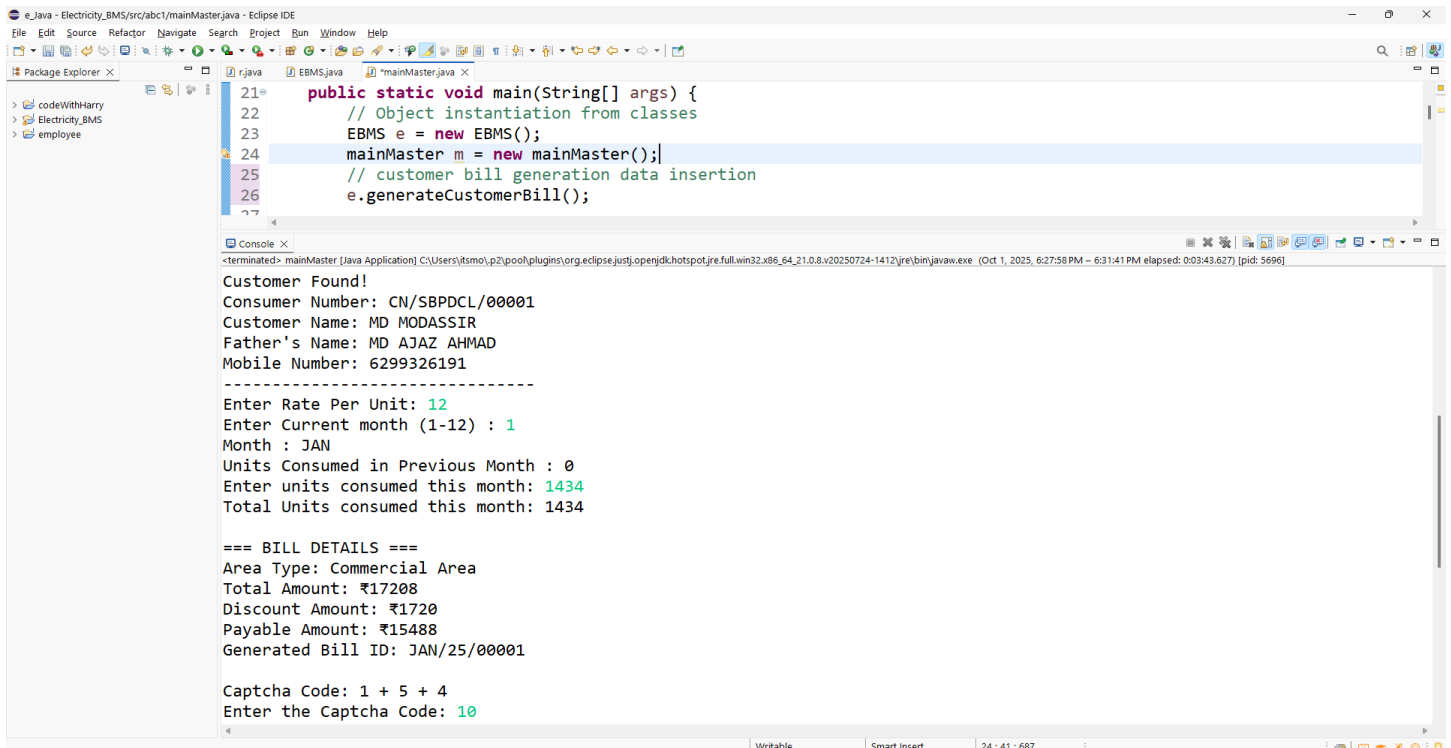
```
20
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // customer bill generation table
26     e.generateCustomerBillTable();
27
28
29
30
```

Console

mainMaster [Java Application] C:\Users\tsmo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 1, 2025, 6:27:58 PM elapsed: 0:00:21) [pid: 5696]

Table Created for Bill Generation

Customer Bill Generation Data Insertion



```
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // customer bill generation data insertion
26     e.generateCustomerBill();
27
```

Console

<terminated> mainMaster [Java Application] C:\Users\tsmo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 1, 2025, 6:27:58 PM - 6:31:41 PM elapsed: 0:03:43.627) [pid: 5696]

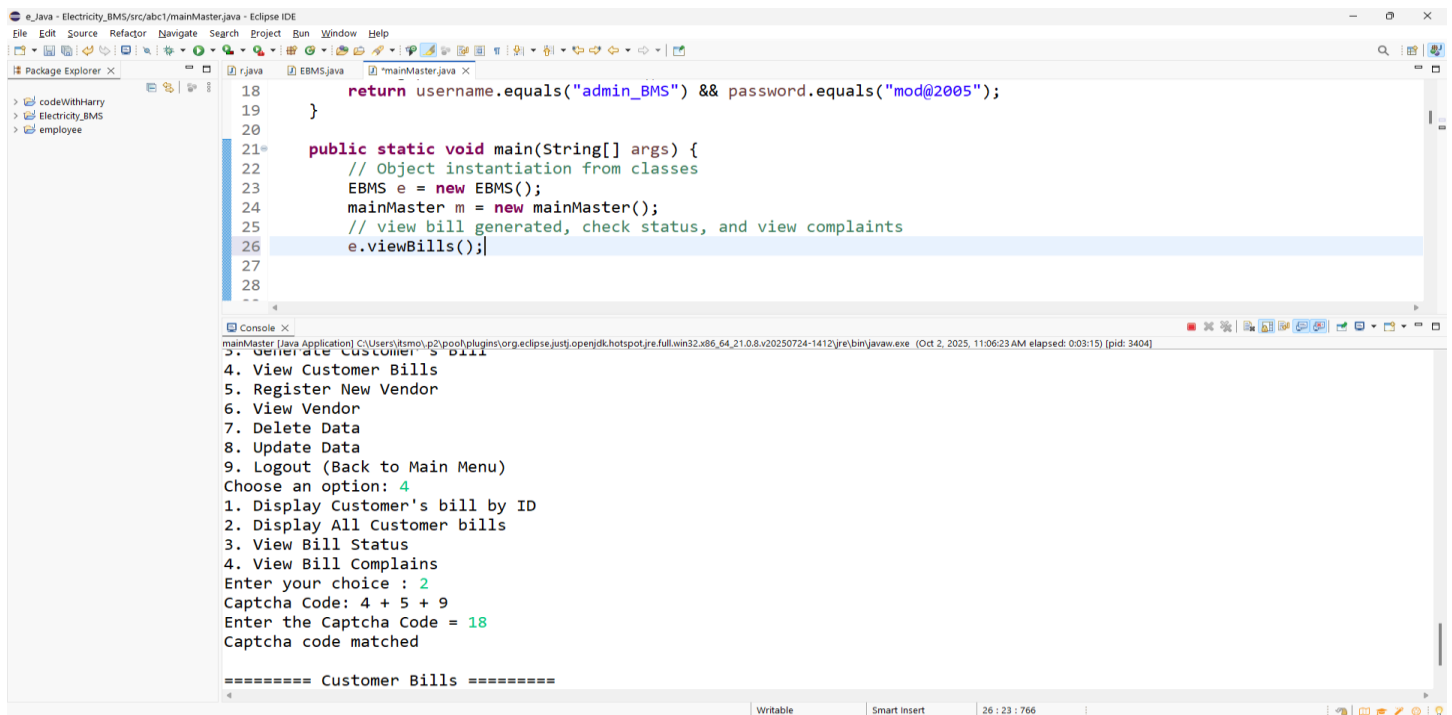
Customer Found!
Consumer Number: CN/SBPDCL/00001
Customer Name: MD MODASSIR
Father's Name: MD AJAZ AHMAD
Mobile Number: 6299326191

Enter Rate Per Unit: 12
Enter Current month (1-12) : 1
Month : JAN
Units Consumed in Previous Month : 0
Enter units consumed this month: 1434
Total Units consumed this month: 1434

=== BILL DETAILS ===
Area Type: Commercial Area
Total Amount: ₹17208
Discount Amount: ₹1720
Payable Amount: ₹15488
Generated Bill ID: JAN/25/00001

Captcha Code: 1 + 5 + 4
Enter the Captcha Code: 10

View Bills Generated, Check Status, and view Complaints



The screenshot shows the Eclipse IDE with the file `mainMaster.java` open. The code includes a login check and a `main` method that instantiates `EBMS` and `mainMaster` objects, then calls `e.viewBills();`. The console output shows a menu with options 1-9, where option 4 (View Bill Complaints) is selected. It then prompts for a choice (2) and a captcha code (18), which matches the displayed captcha (4 + 5 + 9). Finally, it displays a table of customer bills.

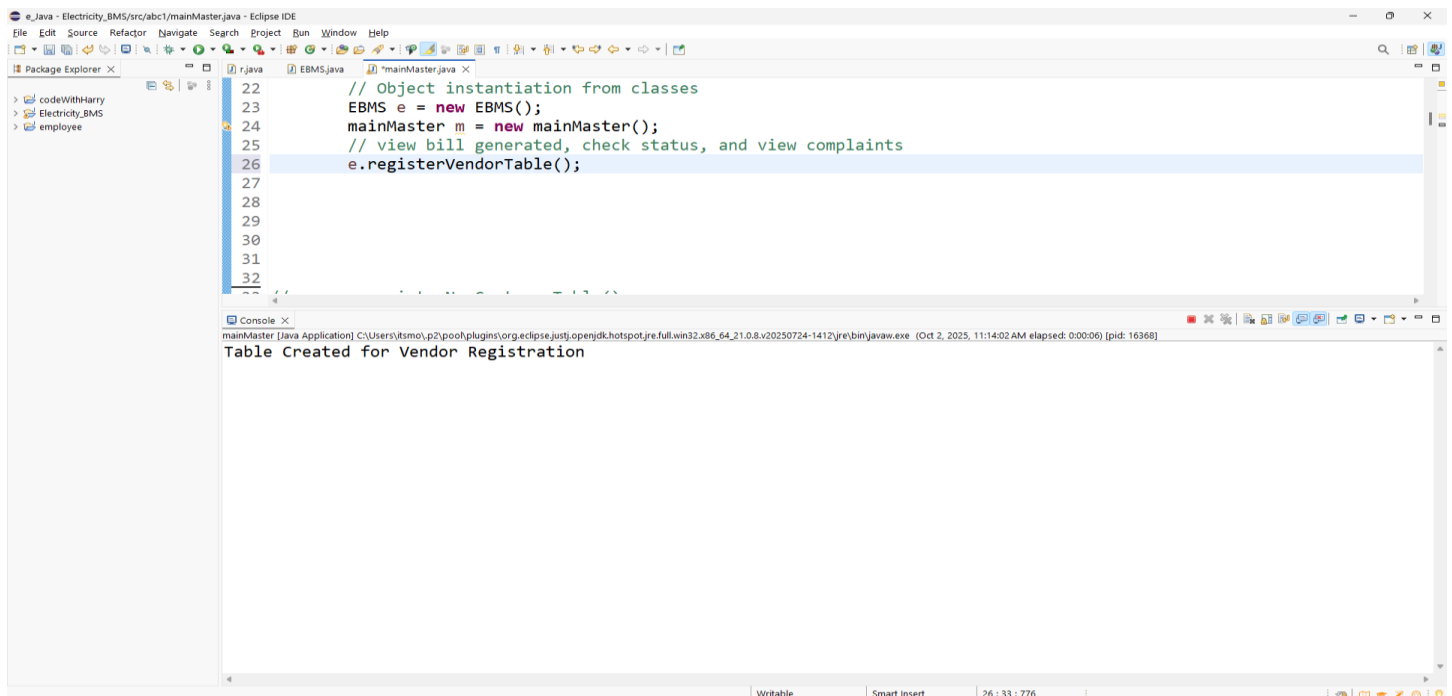
```
18         return username.equals("admin_BMS") && password.equals("mod@2005");
19     }
20
21     public static void main(String[] args) {
22         // Object instantiation from classes
23         EBMS e = new EBMS();
24         mainMaster m = new mainMaster();
25         // view bill generated, check status, and view complaints
26         e.viewBills();
27     }
28 }
```

Console Output:

```
mainMaster [Java Application] C:\Users\tsm01.p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:06:23 AM elapsed: 0:03:15) [pid: 3404]
5. Generate Customer's Bill
4. View Customer Bills
5. Register New Vendor
6. View Vendor
7. Delete Data
8. Update Data
9. Logout (Back to Main Menu)
Choose an option: 4
1. Display Customer's bill by ID
2. Display All Customer bills
3. View Bill Status
4. View Bill Complaints
Enter your choice : 2
Captcha Code: 4 + 5 + 9
Enter the Captcha Code = 18
Captcha code matched

===== Customer Bills =====
```

Vendor Registration Table



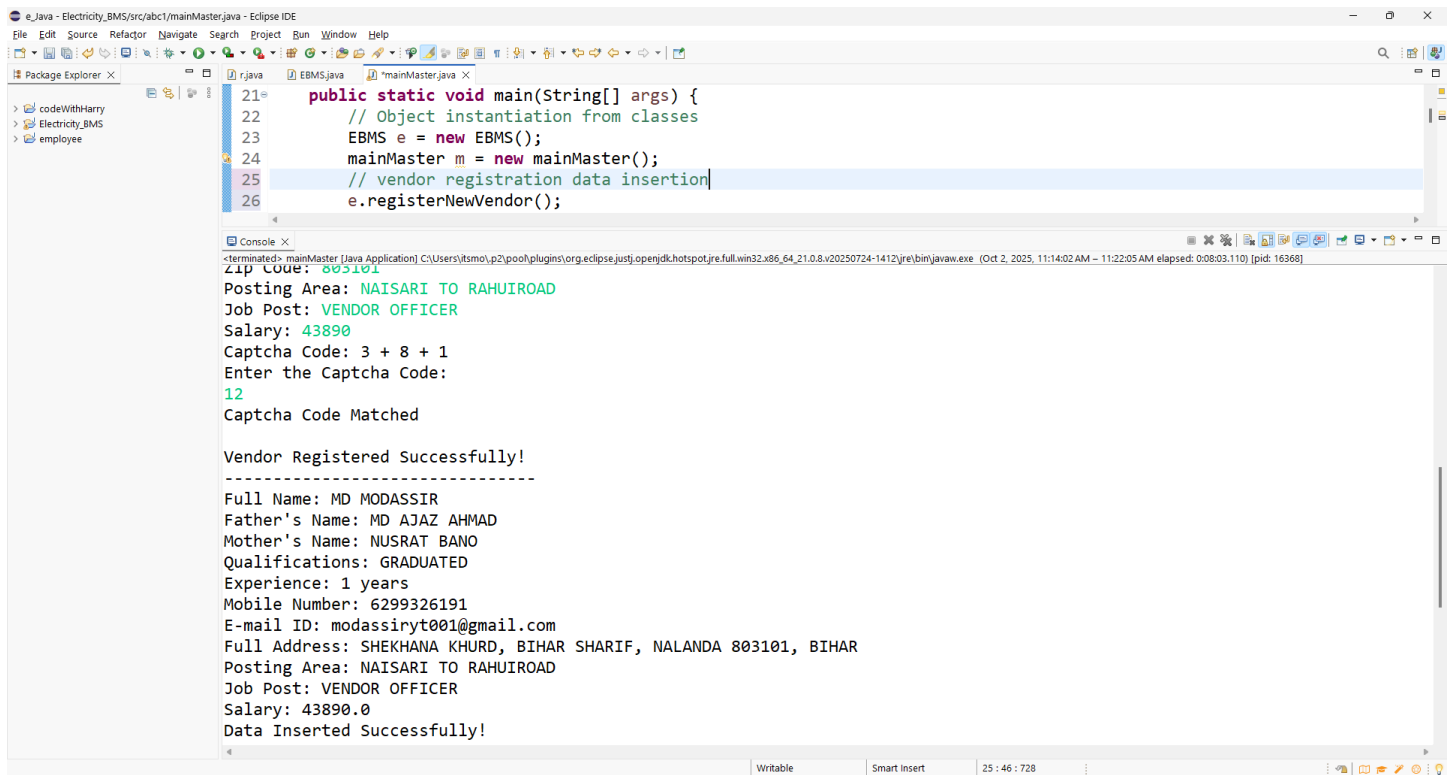
The screenshot shows the Eclipse IDE with the file `mainMaster.java` open. The code includes object instantiation and a call to `e.registerVendorTable();`. The console output shows the message "Table Created for Vendor Registration".

```
22         // Object instantiation from classes
23         EBMS e = new EBMS();
24         mainMaster m = new mainMaster();
25         // view bill generated, check status, and view complaints
26         e.registerVendorTable();
27     }
28 }
```

Console Output:

```
mainMaster [Java Application] C:\Users\tsm01.p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:14:02 AM elapsed: 0:00:06) [pid: 16368]
Table Created for Vendor Registration
```

Vendor Registration Data Insertion



The screenshot shows the Eclipse IDE with a Java project named 'Electricity_BMS'. The 'mainMaster.java' file is open, showing a main method that instantiates an 'EBMS' object and a 'mainMaster' object, then calls 'registerNewVendor()'. The console output shows the registration process, including a captcha challenge and a successful registration message.

```
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // vendor registration data insertion
26     e.registerNewVendor();

```

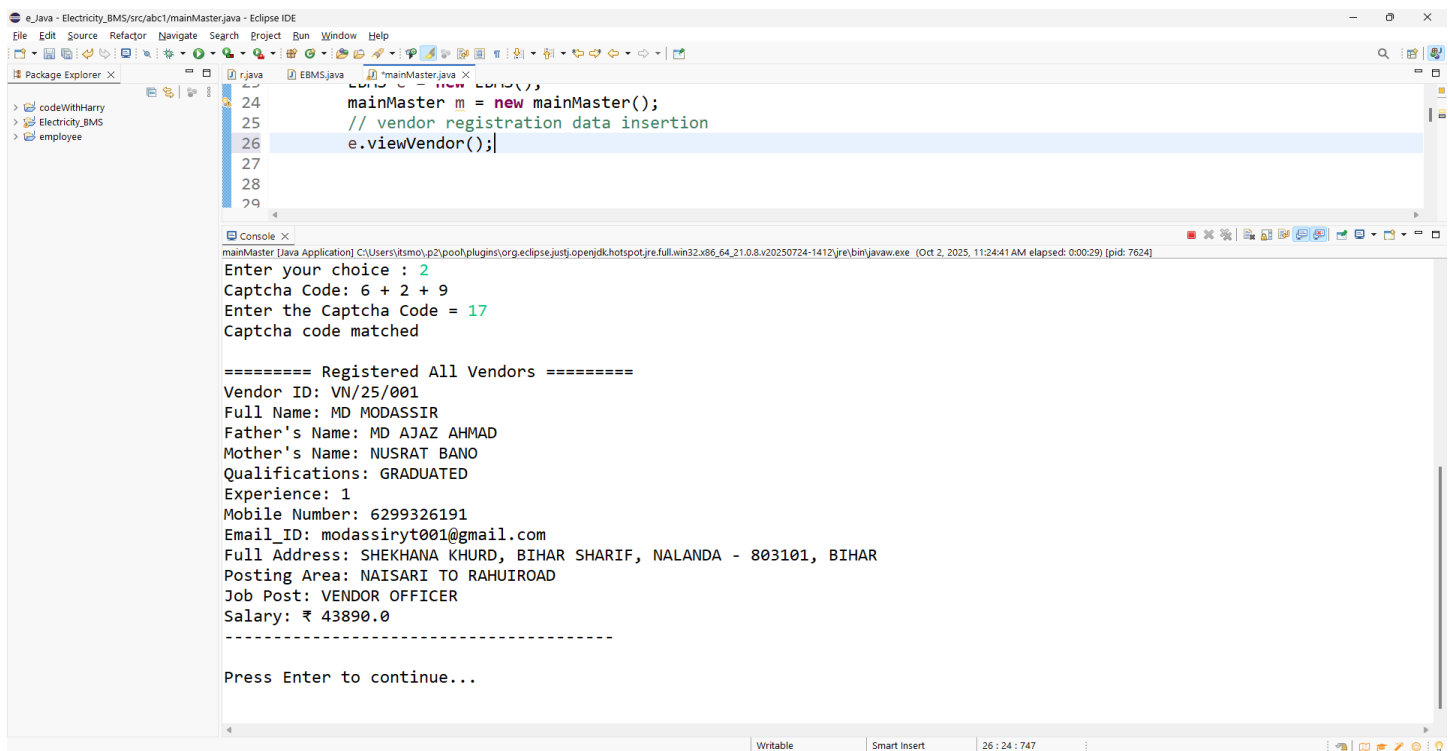
Console Output:

```
<terminated>: mainMaster [Java Application] C:\Users\tsmo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:14:02 AM - 11:22:05 AM elapsed: 0:08:03.110) [pid: 16368]
ZIP CODE: 803101
Posting Area: NAISARI TO RAHUIROAD
Job Post: VENDOR OFFICER
Salary: 43890
Captcha Code: 3 + 8 + 1
Enter the Captcha Code:
12
Captcha Code Matched

Vendor Registered Successfully!
-----
Full Name: MD MODASSIR
Father's Name: MD AJAZ AHMAD
Mother's Name: NUSRAT BANO
Qualifications: GRADUATED
Experience: 1 years
Mobile Number: 6299326191
E-mail ID: modassiry001@gmail.com
Full Address: SHEKHANA KHURD, BIHAR SHARIF, NALANDA 803101, BIHAR
Posting Area: NAISARI TO RAHUIROAD
Job Post: VENDOR OFFICER
Salary: 43890.0
Data Inserted Successfully!

```

View Vendors



The screenshot shows the Eclipse IDE with the 'mainMaster.java' file open. The code now includes a call to 'viewVendor()' in the main method. The console output shows the 'View Vendors' screen, displaying a list of registered vendors and their details.

```
24     EBMS e = new EBMS();
25     mainMaster m = new mainMaster();
26     // vendor registration data insertion
27     e.viewVendor();
28
29

```

Console Output:

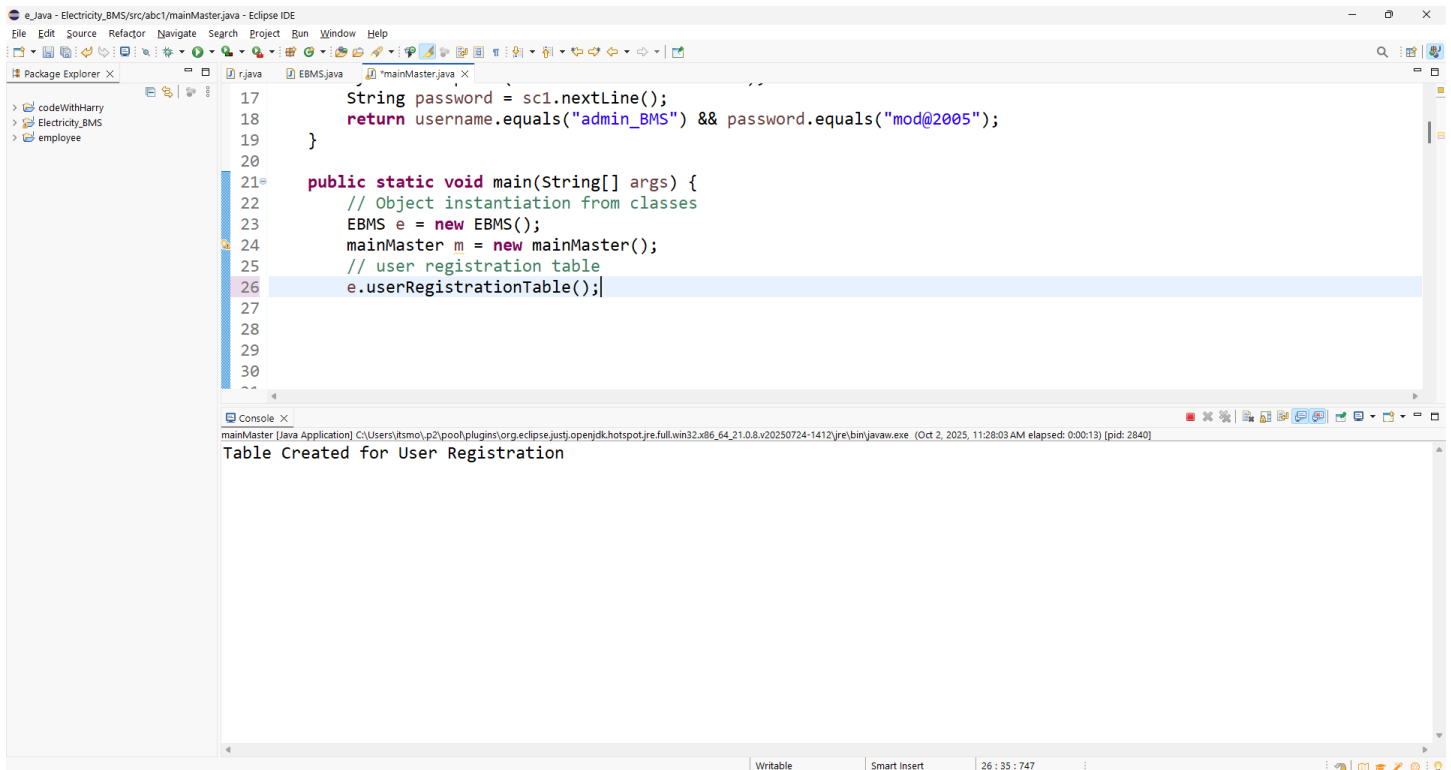
```
mainMaster [Java Application] C:\Users\tsmo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:24:41 AM elapsed: 0:00:29) [pid: 7624]
Enter your choice : 2
Captcha Code: 6 + 2 + 9
Enter the Captcha Code = 17
Captcha code matched

===== Registered All Vendors =====
Vendor ID: VN/25/001
Full Name: MD MODASSIR
Father's Name: MD AJAZ AHMAD
Mother's Name: NUSRAT BANO
Qualifications: GRADUATED
Experience: 1
Mobile Number: 6299326191
Email ID: modassiry001@gmail.com
Full Address: SHEKHANA KHURD, BIHAR SHARIF, NALANDA - 803101, BIHAR
Posting Area: NAISARI TO RAHUIROAD
Job Post: VENDOR OFFICER
Salary: ₹ 43890.0
-----

Press Enter to continue...

```

User Registration Table

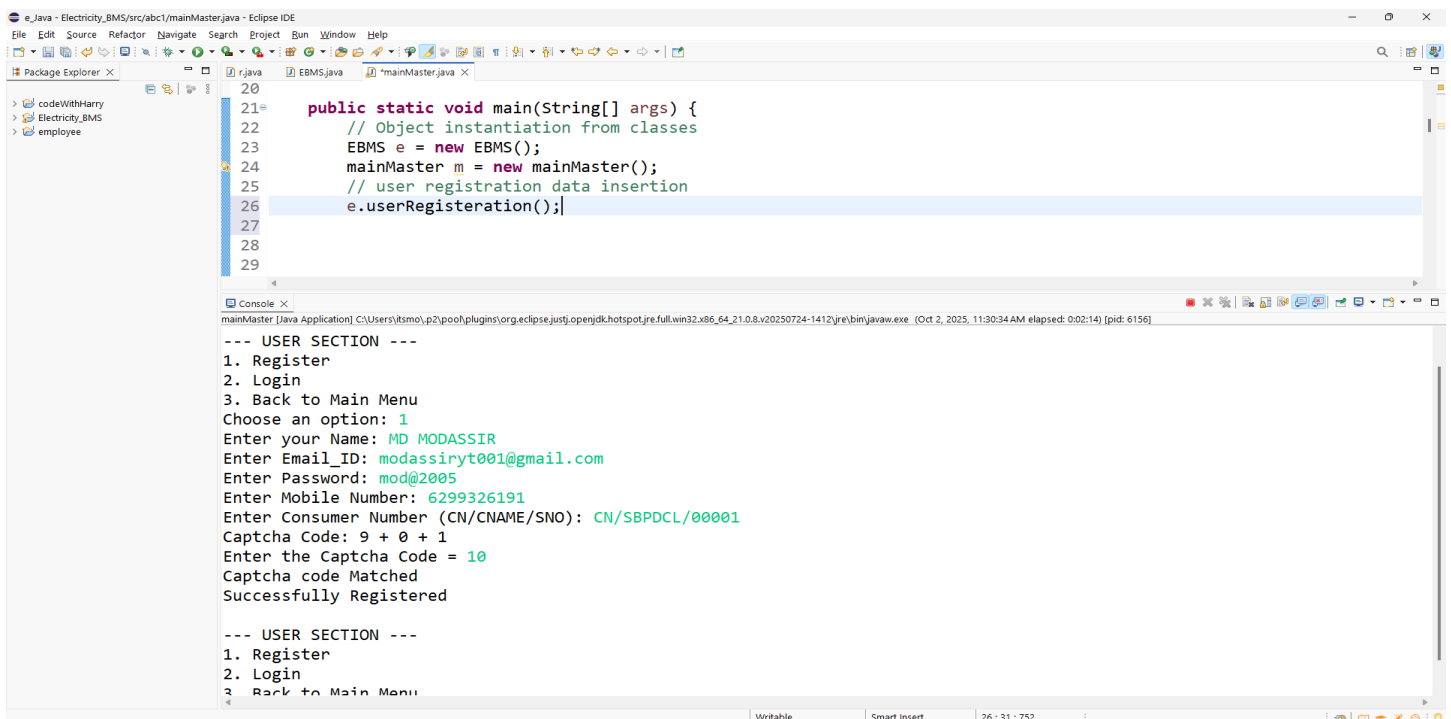


```
17 String password = sc1.nextLine();
18 return username.equals("admin_BMS") && password.equals("mod@2005");
19 }
20
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // user registration table
26     e.userRegistrationTable();
27
28
29
30
31 }
```

mainMaster [Java Application] C:\Users\itsmol.p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:28:03 AM elapsed: 0:00:13) [pid: 2840]

Table Created for User Registration

User Registration Data Insertion



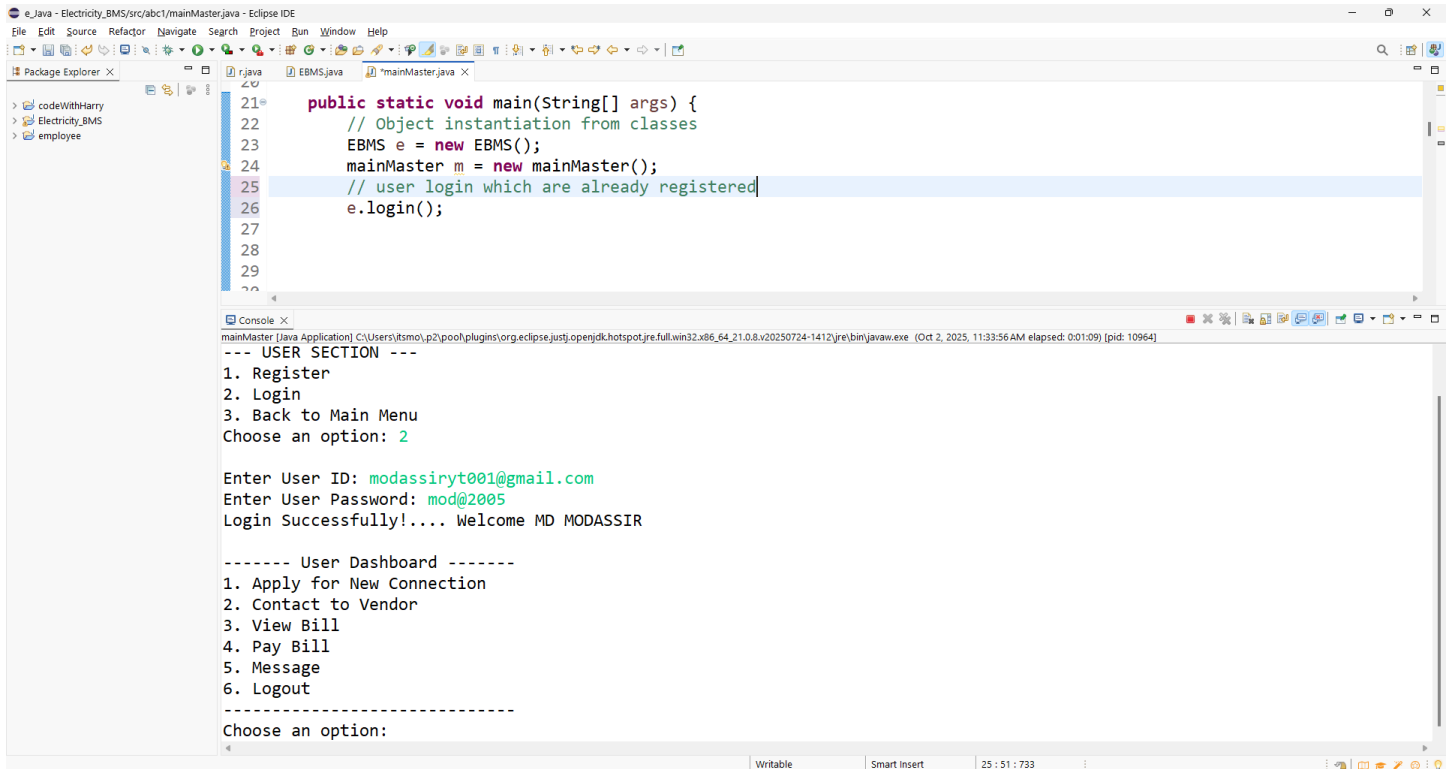
```
20
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // user registration data insertion
26     e.userRegistration();
27
28
29
30
31 }
```

mainMaster [Java Application] C:\Users\itsmol.p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:30:34 AM elapsed: 0:02:14) [pid: 6156]

--- USER SECTION ---
1. Register
2. Login
3. Back to Main Menu
Choose an option: 1
Enter your Name: MD MODASSIR
Enter Email_ID: modassiry001@gmail.com
Enter Password: mod@2005
Enter Mobile Number: 6299326191
Enter Consumer Number (CN/CNAME/SNO): CN/SBPDCL/00001
Captcha Code: 9 + 0 + 1
Enter the Captcha Code = 10
Captcha code Matched
Successfully Registered

--- USER SECTION ---
1. Register
2. Login
3. Back to Main Menu

User Login which are already Registered



The screenshot shows the Eclipse IDE with the file `mainMaster.java` open. The code in the editor is as follows:

```
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // user login which are already registered
26     e.login();
27
28
29 }
```

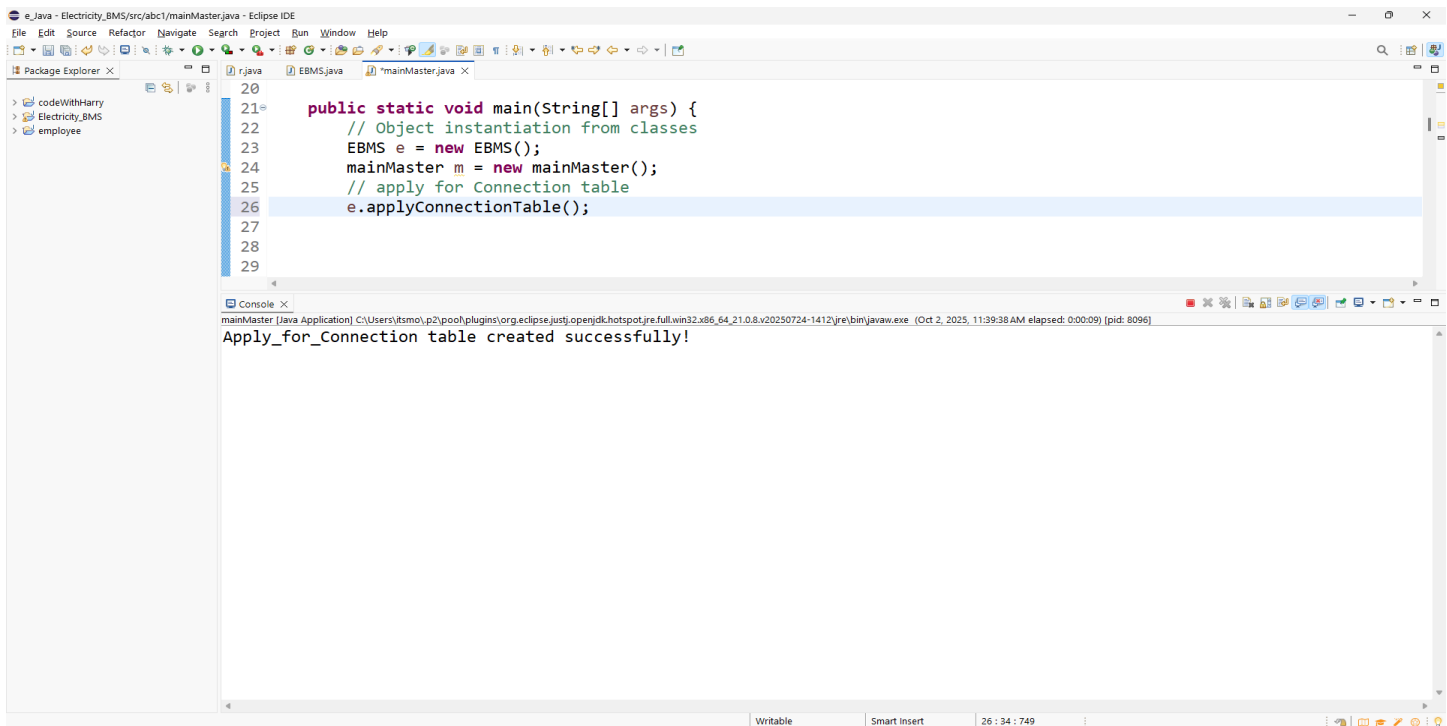
The console output shows the following sequence of events:

```
mainMaster [Java Application] C:\Users\tsmo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:33:56 AM elapsed: 0:01:09) [pid: 10964]
---- USER SECTION ----
1. Register
2. Login
3. Back to Main Menu
Choose an option: 2

Enter User ID: modassiry001@gmail.com
Enter User Password: mod@2005
Login Successfully!.... Welcome MD MODASSIR

----- User Dashboard -----
1. Apply for New Connection
2. Contact to Vendor
3. View Bill
4. Pay Bill
5. Message
6. Logout
-----
Choose an option:
```

Apply for Connection Table



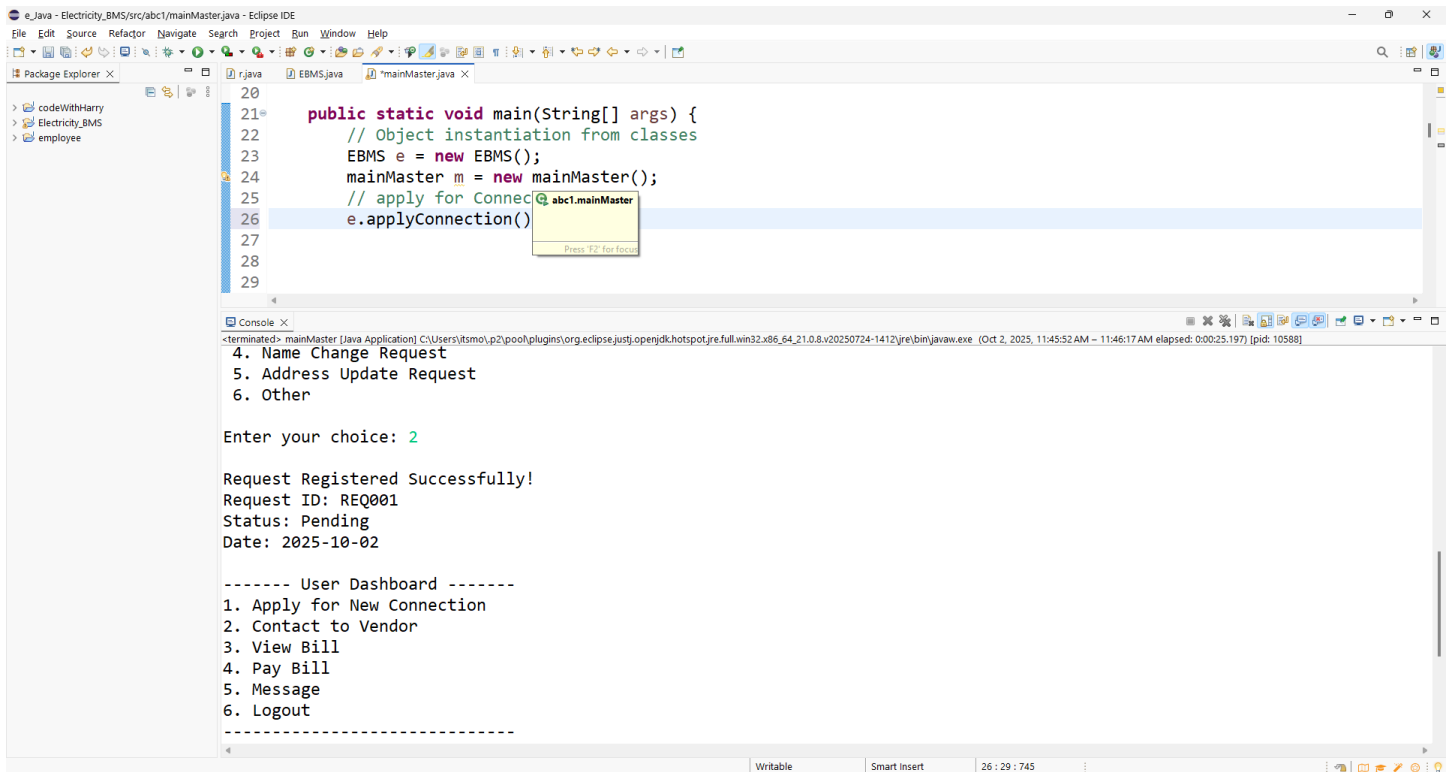
The screenshot shows the Eclipse IDE with the file `mainMaster.java` open. The code in the editor is as follows:

```
20
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // apply for Connection table
26     e.applyConnectionTable();
27
28
29 }
```

The console output shows the following sequence of events:

```
mainMaster [Java Application] C:\Users\tsmo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:39:38 AM elapsed: 0:00:09) [pid: 8096]
Apply_for_Connection table created successfully!
```

Apply for Connection (user)



```
20
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // apply for Connection
26     e.applyConnection();
27
28
29
```

Console Output:

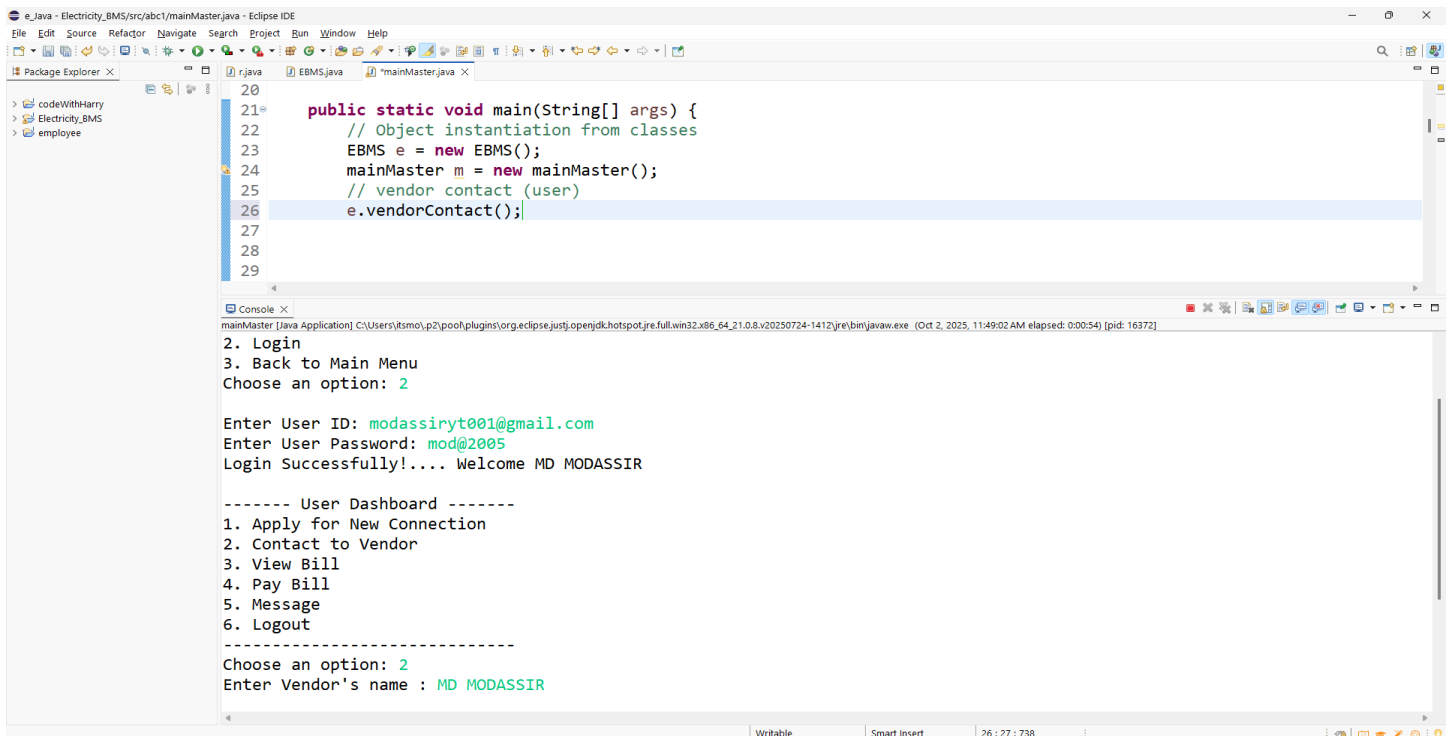
```
<terminated> mainMaster [Java Application] C:\Users\itsmo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:45:52 AM - 11:46:17 AM elapsed: 0:00:25.197) [pid: 10588]
4. Name Change Request
5. Address Update Request
6. Other

Enter your choice: 2

Request Registered Successfully!
Request ID: REQ001
Status: Pending
Date: 2025-10-02

----- User Dashboard -----
1. Apply for New Connection
2. Contact to Vendor
3. View Bill
4. Pay Bill
5. Message
6. Logout
-----
```

Vendor Contact (user)



```
20
21 public static void main(String[] args) {
22     // Object instantiation from classes
23     EBMS e = new EBMS();
24     mainMaster m = new mainMaster();
25     // vendor contact (user)
26     e.vendorContact();
27
28
29
```

Console Output:

```
mainMaster [Java Application] C:\Users\itsmo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:49:02 AM elapsed: 0:00:54) [pid: 16372]
2. Login
3. Back to Main Menu
4. Choose an option: 2

Enter User ID: modassiry001@gmail.com
Enter User Password: mod@2005
Login Successfully!.... Welcome MD MODASSIR

----- User Dashboard -----
1. Apply for New Connection
2. Contact to Vendor
3. View Bill
4. Pay Bill
5. Message
6. Logout
-----

Choose an option: 2
Enter Vendor's name : MD MODASSIR
```

View Bill (user)

```
mainMaster.java
22 // Object instantiation from classes
23 EBMS e = new EBMS();
24 mainMaster m = new mainMaster();
25 // View Bill (user)
26 e.viewBill();
27
28
29
30
31
```

Console Output:

```
mainMaster [Java Application] C:\Users\tsmol.p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:50:58 AM elapsed: 0:01:06) [pid: 11736]
Choose an option: 2

Enter User ID: modassiry001@gmail.com
Enter User Password: mod@2005
Login Successfully!.... Welcome MD MODASSIR

----- User Dashboard -----
1. Apply for New Connection
2. Contact to Vendor
3. View Bill
4. Pay Bill
5. Message
6. Logout

Choose an option: 3
1. View Consumption Details
2. View Bill Payment History
Choose an option : 2
```

Pay Bill (user)

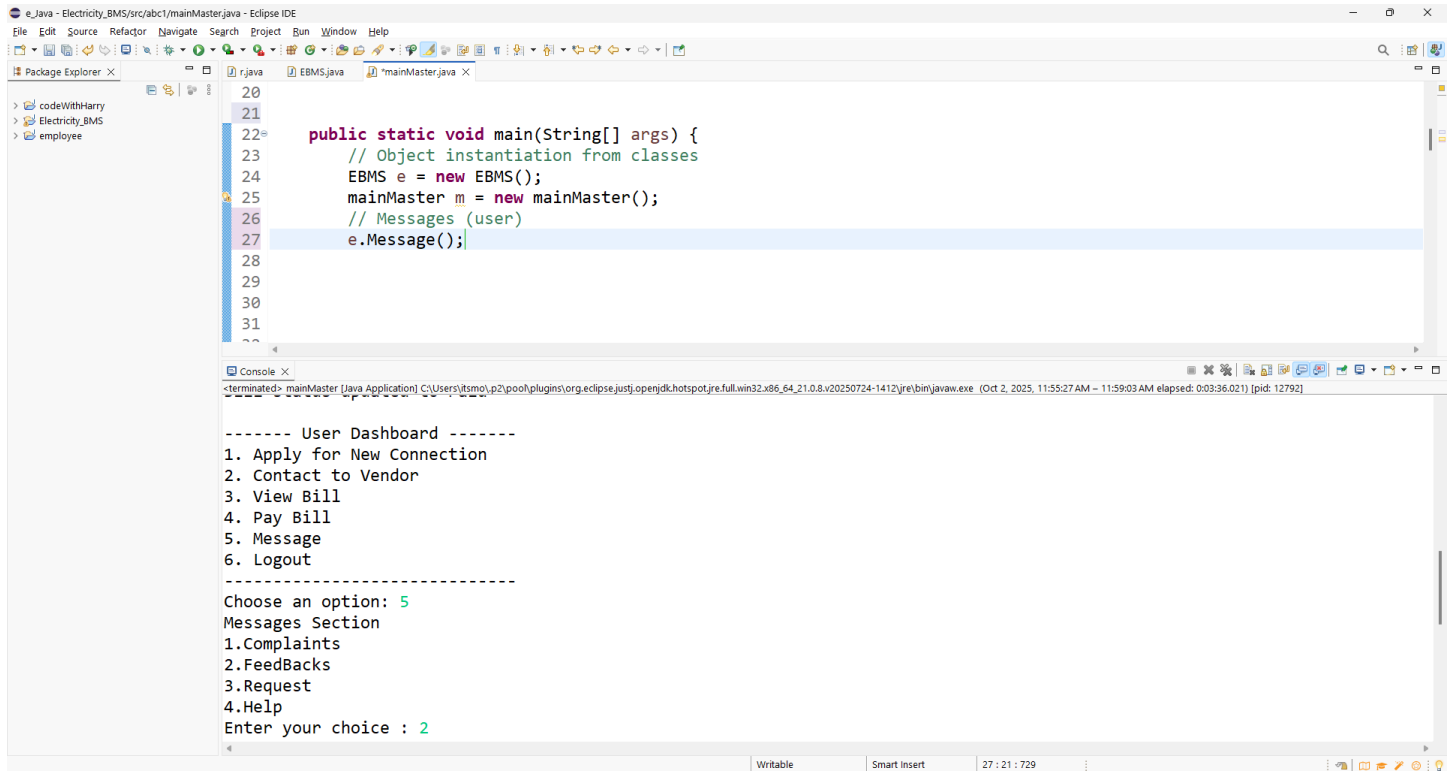
```
mainMaster.java
20
21
22 public static void main(String[] args) {
23 // Object instantiation from classes
24 EBMS e = new EBMS();
25 mainMaster m = new mainMaster();
26 // pay Bill (user)
27 e.payBill();
28
29
30
31
32
```

Console Output:

```
mainMaster [Java Application] C:\Users\tsmol.p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:55:27 AM elapsed: 0:01:23) [pid: 12792]
4. Pay bill
5. Message
6. Logout

-----
Choose an option: 4
Enter Customer's ID : JAN/25/00001
Amount : 14068
Status : Pending
Payment Method
1. ATM-card
2. UPI
Choose option: 1
Enter ATM-card No (CCV/4 digit ATM) : CCV/6151
Enter the Amount : 14068
Payment Successful
Bill status updated to Paid
```

Messages (user)



The screenshot shows the Eclipse IDE with the following components:

- Package Explorer:** Shows a project named 'Electricity_BMS' with sub-packages 'codeWithHarry', 'Electricity_BMS', and 'employee'.
- Editor:** Displays the file 'mainMaster.java' with the following code:

```
20
21
22 public static void main(String[] args) {
23     // Object instantiation from classes
24     EBMS e = new EBMS();
25     mainMaster m = new mainMaster();
26     // Messages (user)
27     e.Message();
28
29
30
31
32 }
```
- Console:** Shows the output of the program:

```
<terminated> mainMaster (Java Application) C:\Users\itsmo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 2, 2025, 11:55:27 AM - 11:59:03 AM elapsed: 0:03:36.021) [pid: 12792]

----- User Dashboard -----
1. Apply for New Connection
2. Contact to Vendor
3. View Bill
4. Pay Bill
5. Message
6. Logout

Choose an option: 5
Messages Section
1.Complaints
2.FeedBacks
3.Request
4.Help
Enter your choice : 2
```

4.5 **Maintenance**

The maintenance phase for the Electricity Bill Management System establishes a comprehensive framework for ensuring continuous system reliability and performance. Based on the successfully implemented modules—including User Registration, Bill Generation, Payment Processing, Vendor Management, and Connection Handling—a structured maintenance schedule is defined. This includes daily health monitoring, weekly data integrity checks, and monthly performance optimization. Specific procedures address each functional module, with regular validation of Java business logic, MySQL database integrity, and user interface functionality. The maintenance plan incorporates robust backup strategies, security protocols, and a future enhancement roadmap to ensure the system evolves with changing requirements while maintaining the high reliability demonstrated during testing.

4.5.1 **Module-Specific Maintenance**

User Registration:

- Monthly validation rule updates
- Password encryption integrity checks
- Role-based access verification
- Bill Generation & Payment:
- Quarterly tariff calculation validation
- Payment status trigger testing
- Bill PDF generation Checks

Vendor Management:

- Vendor performance reviews
- Assignment logic verification
- Salary calculation accuracy

4.5.2 **Error Recovery**

Common Issues:

- User login failure resolution
- Bill calculation errors
- Payment processing failures

Future Roadmap and Enhancement

Manually managing all operational tasks is a significant challenge for any organization. To address this, an automated management system is essential for efficiently handling product and institutional data.

The proposed system is designed for adaptability. It can be deployed in similar organizations with minimal customization to meet specific hardware, software, or evolving staff requirements.

The architecture is inherently flexible, allowing for future enhancements in several key areas:

- **Scalable Functionality:** The system has a built-in capacity for modifying hierarchies and adding new capabilities.
- **Adaptable Interface:** Dynamic screens can be implemented to meet changing operational needs.
- **Future-Proof Design:** The system is structured to accommodate further modifications easily, ensuring it remains up-to-date.
- **Broad Applicability:** With minor adjustments, the system can be repurposed for use in other similar environments.

This is it, Thank You.